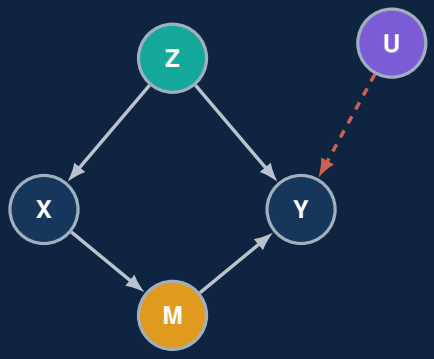


ADVANCED ARTIFICIAL INTELLIGENCE

Probabilistic Graphical Models and Causal Inference

Session 7

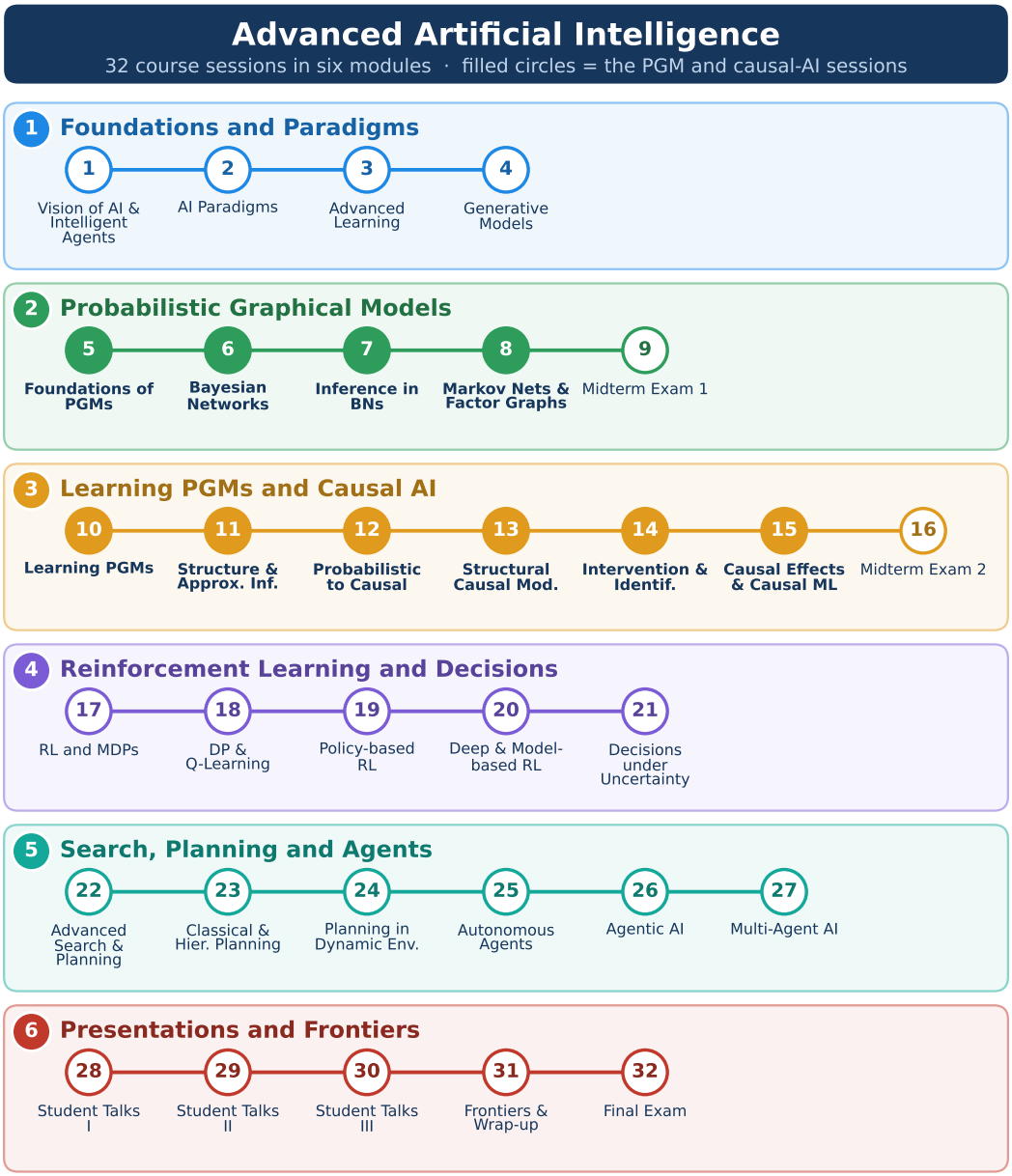
Inference: Variable Elimination, Belief Propagation and MAP

Lecture Notes

Instructor: Prof. M. Arashi

About these notes

These notes cover Session 7 of the course *Advanced Artificial Intelligence*, part of its probabilistic-graphical-model and causal-AI block (course sessions 5–15). The session follows a fixed structure: a roadmap and learning objectives, a "you are here" course map, the core theory organized in parts, a dedicated part on mathematical statistics, fully worked numerical and analytical examples, and a part connecting the topic to intelligent agents, agentic AI, reinforcement learning, multi-agent systems and the other sessions of the course. Every number is reproduced by the companion Python scripts (`python/s07_examples.py` for the numbers, `python/s07_figures.py` for the figures), and `questions/s07_questions.tex` contains ten questions with complete answers.



The Advanced Artificial Intelligence course. Filled circles mark the PGM and causal-AI sessions.

Contents

Session 7 Inference: Variable Elimination, Belief Propagation and MAP **1**

7.1 The Inference Problem . . . 3
7.2 Factors: The Currency of Computation . . . 4
7.3 Elimination Order Matters: Treewidth . . . 6
7.4 From Variable Elimination to Messages on Trees . . . 8
7.5 Chains in Time: HMM Filtering and Smoothing . . . 10
7.6 Clique Trees (Junction Trees) . . . 12
7.7 Loopy Belief Propagation . . . 13
7.8 MAP and Max-Product . . . 13
7.9 Viterbi Decoding in HMMs . . . 14
7.10 MAP versus Marginal MAP versus Max-Marginals . . . 16
7.11 Exact Inference in Linear-Gaussian Networks . . . 17
7.12 Decision Theory: Which Point Estimate? . . . 18
7.13 The Kalman Filter is Belief Propagation . . . 20
7.14 Worked Examples . . . 21
7.15 Inference as the Reasoning Step of an Agent . . . 24
7.16 Summary . . . 29
7.17 Appendix: Full Inductive Proof of VE Correctness . . . 30

SESSION 7 ADVANCED ARTIFICIAL INTELLIGENCE

Inference: Variable Elimination, Belief Propagation and MAP

Exact inference, variable elimination, belief propagation and MAP inference

Advanced Artificial Intelligence · Probabilistic Graphical Models and Causal Inference

Instructor: Prof. M. Arashi

PART I
Exact Inference & VE
Queries, factors, variable elimination, treewidth and hardness.

PART II
Belief Propagation
Sum-product on trees, HMM filtering, clique trees, loopy BP.

PART III
MAP Inference
Max-product, Viterbi decoding, MAP vs marginal MAP.

PART IV
Mathematical Statistics
Gaussian conditioning, Bayes estimators, Kalman filter.

PART V
AI Connections
Agent reasoning, POMDP beliefs, decoding, multi-agent messages.

LEARNING OBJECTIVES

- Distinguish conditional-probability, evidence, MAP/MPE and marginal-MAP queries, and state why exact inference is NP-hard in general.
- Run variable elimination by hand with factor products and sum-outs, and bound its cost by the induced width of the elimination order.
- Derive the sum-product message equations, prove that belief propagation is exact on trees, and recognise the HMM forward algorithm, clique trees and loopy BP as instances of the same idea.
- Replace sums by maxima (max-product, Viterbi) and explain why MAP, marginal MAP and max-marginals disagree.
- Prove the Gaussian conditioning formulas (exact inference in linear-Gaussian networks, the Kalman step) and justify mean, median and MAP as Bayes estimators.
- Connect inference to the reasoning step of agents, POMDP belief updates, sequence decoding, planning and distributed multi-agent inference.

Where we are. Session 5 gave us conditional independence and factorization; Session 6 turned them into Bayesian networks and taught us to *read* independences with d-separation. Now we *ask the model questions*: posteriors, likelihoods and most probable explanations, exactly and efficiently. Session 8 runs the same algorithms on undirected models, Sessions 10–11 use inference inside learning and replace it by sampling on dense graphs; later, inference returns as belief tracking (course

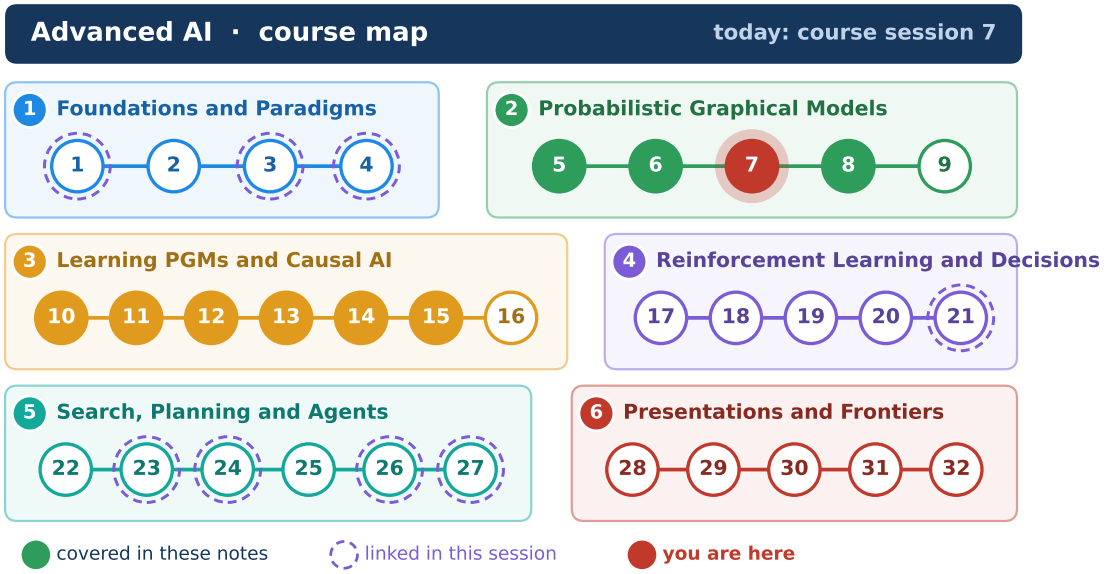


Figure 7.1. Where this session sits in the Advanced AI course (course session 7). Dashed purple rings mark the sessions this chapter links to explicitly.

session 21), decoding (course sessions 3–4), planning (course sessions 23–24) and communication between agents (course session 27).

OPENING MOTIVATION

A doctor enters a patient's observed symptoms into a diagnostic Bayesian network and wants $P(\text{Disease} \mid \text{Symptoms observed})$. A streaming service has built a network relating latent taste clusters to observed ratings and wants $P(\text{likes genre} \mid \text{ratings so far})$. In both cases we have a Bayesian network — a graph plus local conditional probability tables, exactly what Session 6 gave us — but we have not yet asked the practical question: *how do we actually compute an answer?* The naive approach, summing over the full joint distribution, is exactly the $O(2^n)$ catastrophe Session 5 opened with. Today's algorithm, variable elimination, shows how to answer such queries *exactly*, while doing work that scales with the graph's structure rather than blindly with 2^n.

Healthcare

A diagnostic network computes $P(\text{disease} \mid \text{test results})$ and the *most probable* joint explanation of all symptoms; the next test is chosen by how much it would change the posterior.

Robotics

A robot's belief over its position is an HMM filtering distribution, updated by one predict–correct step per sensor reading: exact inference running in real time.

LLM agents

An agent that monitors tools decodes the most probable sequence of hidden service states (Viterbi), and queries a Bayesian-network tool for diagnostic posteriors before acting.

PART I

Exact Inference and Variable Elimination

Pushing sums inside products — and paying for it in treewidth

PART I · EXACT INFERENCE

7.1 The Inference Problem

Definition 7.1 (Probabilistic query)**.** Given a Bayesian network over variables $\mathbf{X}$, a query variable set $Y \subset \mathbf{X}$, and observed evidence $E = e$ for some disjoint subset $E \subset \mathbf{X}$, *inference* means computing

$$P(y \mid e) = \frac{P(y, e)}{P(e)} = \frac{\sum_w P(y, e, w)}{\sum_{y', w} P(y', e, w)}, \qquad W = \mathbf{X} \setminus (Y \cup E), \tag{7.1}$$

where W are the remaining ("nuisance") variables summed out.

Computing $P(y, e)$ by summing the full joint table over all values of W is the naive algorithm; for $|W|$ binary variables it costs $O(2^{|W|})$. The Bayesian network factorization $P(\mathbf{x}) = \prod_i P(x_i \mid \mathrm{pa}(x_i))$ from Session 6 gives us a smarter option: because the joint is a *product of small local factors*, we can push summations inside the product — summing out each nuisance variable as early as possible, working only with the factors that actually mention it.

7.1.1 Four kinds of query

Conditional probabilities are not the only question we can ask of a joint distribution. Four queries recur throughout the course; (i)–(ii) need sums (sum-product), (iii) maxima (max-product) and (iv) both.

Definition 7.2 (Query types)**.** With evidence e and non-evidence variables split into Y and W: (i) the *conditional query* $P(Y \mid e)$; (ii) the *probability of evidence* $P(e) = \sum_{y,w} P(y, w, e)$, the normaliser of (i) and the *likelihood* of the data; (iii) the *MAP* or *MPE* query $\arg\max_{y,w} P(y, w \mid e)$, one joint assignment of *all* non-evidence variables; (iv) the *marginal MAP* query $\arg\max_y \sum_w P(y, w \mid e)$ over a subset Y.

Terminology varies: older Bayesian-network texts call (iii) "MPE" and (iv) "MAP"; we follow Koller & Friedman (2009): MAP = MPE for (iii), "marginal MAP" for (iv).

In the worst case (i)–(ii) are #P-hard, (iii) is NP-hard and (iv) is $\mathrm{NP}^{\mathrm{PP}}$-complete, harder than both (Koller & Friedman 2009, Secs. 9.1 and 13.1; the first claim is proved below).

7.1.2 Exact inference is NP-hard

Theorem 7.1 (Hardness of exact inference; Cooper 1990)**.** *Deciding, for a Bayesian network with binary variables and a variable X, whether $P(X = 1) > 0$ is NP-complete. Consequently computing $P(X = 1)$ exactly is NP-hard (indeed #P-hard).*

Proof sketch. A full assignment of positive probability is an NP certificate. For hardness, given a 3-CNF formula over $q_1, \dots, q_n$ with clauses $c_1, \dots, c_m$, build roots Q_i with $P(Q_i = 1) = \frac{1}{2}$, a deterministic OR node C_j per clause (parents: its three variables), and a chain of deterministic AND nodes $A_1 = C_1$, $A_j = A_{j-1} \wedge C_j$ (in-degree ≤ 3, size $O(n + m)$). For every $\mathbf{q}$, $A_m = 1$ iff the formula holds, so $P(A_m = 1) = \#\mathrm{SAT}/2^n$, positive iff satisfiable, and computing it counts solutions. □

Even approximating posteriors to fixed absolute precision is NP-hard (Dagum & Luby 1993). The way out is *structure*: the cost of exact inference is governed by one graph parameter, the treewidth.

PART I · EXACT INFERENCE

7.2 Factors: The Currency of Computation

Definition 7.3 (Factor)**.** A *factor* $\phi(\mathbf{Y})$ over a scope $\mathbf{Y} \subseteq \mathbf{X}$ is any function assigning a non-negative real number to each joint assignment of $\mathbf{Y}$. Every conditional probability table $P(x_i \mid \mathrm{pa}(x_i))$ is a factor with scope $\{X_i\} \cup \mathrm{pa}(X_i)$; evidence is incorporated by simply restricting a factor to the observed value.

Two operations on factors are all variable elimination (VE) ever needs:

- **Factor product**: $(\phi_1 \cdot \phi_2)(\mathbf{Y}_1 \cup \mathbf{Y}_2) = \phi_1(\mathbf{y}_1)\phi_2(\mathbf{y}_2)$, matching shared variables.
- **Sum-out (marginalization)**: $\big(\sum_Z \phi\big)(\mathbf{Y} \setminus Z) = \sum_z \phi(\mathbf{y} \setminus Z, z)$.

A third, trivial operation is **reduction**: $\phi[E = e]$ keeps only the rows consistent with the evidence and drops E from the scope. The algebraic fact behind everything in this session is the *distributive law*: if ϕ_1 does not mention Z, then

$$\sum_z \phi_1(\mathbf{y}_1)\, \phi_2(\mathbf{y}_2, z) = \phi_1(\mathbf{y}_1) \sum_z \phi_2(\mathbf{y}_2, z). \tag{7.2}$$

7.2.1 The Variable Elimination Algorithm

Algorithm 7.1 Variable Elimination for computing $P(Y, e)$

1: Initialize the factor set Φ with every CPT of the network, each restricted to evidence $E = e$.
2: Choose an elimination order σ over the nuisance variables $W = \mathbf{X} \setminus (Y \cup E)$.
3: **for** each variable Z in order σ **do**
4: Collect all factors in Φ whose scope contains Z; call their product ψ.
5: Compute $\tau = \sum_Z \psi$ (sum Z out of ψ).
6: Remove the collected factors from Φ; add τ to Φ.
7: **end for**
8: Multiply all remaining factors in Φ to get an (unnormalized) factor over Y; normalize to sum to 1.

Figure 7.2 shows one elimination step in isolation: every factor that mentions the variable being eliminated is multiplied together first, and only then is the summation performed — this order of operations (multiply the relevant pieces, *then* sum) is the entire trick that avoids ever materializing the full joint table.

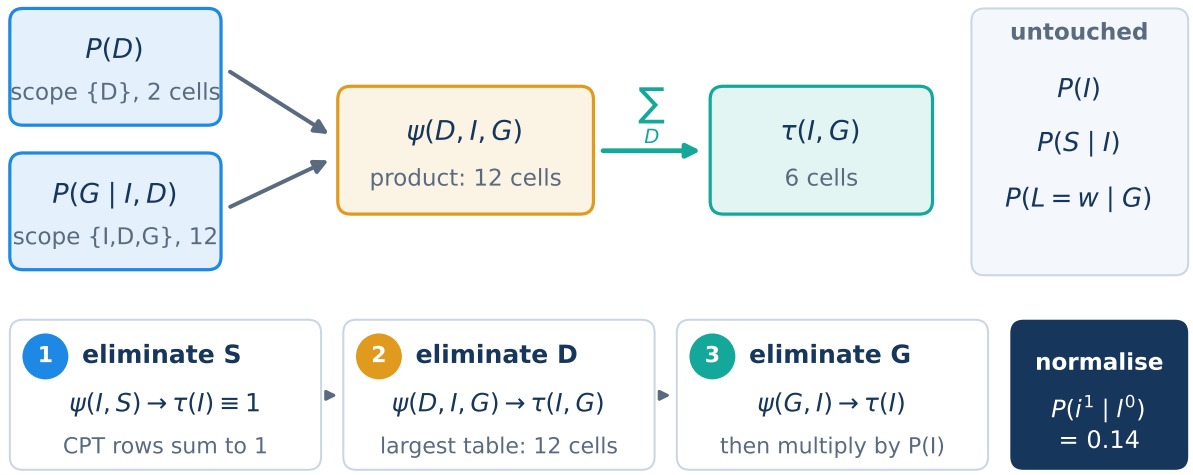


Figure 7.2. One step of variable elimination: combine every factor mentioning the eliminated variable, then marginalize. Bottom: the three steps of the order (S, D, G) for the query $P(I \mid L = \text{weak})$ in the student network (Worked Example 7.1).

Proposition 7.1 (Correctness of variable elimination)**.** *The value returned by variable elimination equals $P(y, e)$ exactly, for any elimination order.*

Proof idea. This follows from two facts: (1) summation distributes over multiplication (7.2), so pushing a sum for Z inside the product of only the factors that mention Z, and leaving the rest outside, changes nothing algebraically; (2) repeating this for every variable in W, one at a time, is exactly the process of computing $\sum_w \prod_i P(x_i \mid \text{pa}(x_i))$ — the definition of $P(y, e)$ — just grouped efficiently. A full inductive proof is in the appendix (Section 7.17). □

Worked Example 7.1 (The student network, with numbers)**.** Use the student network of Session 6 ($D \to G \leftarrow I$, $I \to S$, $G \to L$) with the CPTs of Koller & Friedman (2009) and

Session 6's value labels: d^0/d^1 = easy/hard course, i^0/i^1 = low/high intelligence, g^1, g^2, g^3 = grades A, B, C, s^0/s^1 = low/high SAT, l^0/l^1 = weak/strong letter. The CPTs are $P(d^1) = 0.4$, $P(i^1) = 0.3$; $P(G \mid I, D)$ with rows $(P(g^1), P(g^2), P(g^3)) = (0.3, 0.4, 0.3)$ for (i^0, d^0), $(0.05, 0.25, 0.7)$ for (i^0, d^1), $(0.9, 0.08, 0.02)$ for (i^1, d^0), $(0.5, 0.3, 0.2)$ for (i^1, d^1); $P(s^1 \mid i^0) = 0.05$, $P(s^1 \mid i^1) = 0.8$; $P(l^0 \mid g^1) = 0.1$, $P(l^0 \mid g^2) = 0.4$, $P(l^0 \mid g^3) = 0.99$. Compute $P(i^1 \mid l^0)$ (a high-intelligence student, given a weak letter) by VE with order (S, D, G).

Solution. The evidence $L = l^0$ reduces $P(L \mid G)$ to the factor $P(l^0 \mid G) = (0.1, 0.4, 0.99)$ over G.

1 Eliminate S: $\tau_0(I) = \sum_s P(s \mid I) = 1$ for both values of I (each CPT row sums to one).

2 Eliminate D: $\tau_1(I, G) = \sum_d P(d)P(G \mid I, d)$, using $P(d^0) = 1 - 0.4 = 0.6$:

$$\begin{aligned}\tau_1(i^0, \cdot) &= 0.6\,(0.3,\ 0.4,\ 0.3) + 0.4\,(0.05,\ 0.25,\ 0.7)\\ &= (0.18 + 0.02,\ 0.24 + 0.10,\ 0.18 + 0.28) = (0.20,\ 0.34,\ 0.46),\\ \tau_1(i^1, \cdot) &= 0.6\,(0.9,\ 0.08,\ 0.02) + 0.4\,(0.5,\ 0.3,\ 0.2)\\ &= (0.54 + 0.20,\ 0.048 + 0.12,\ 0.012 + 0.08) = (0.74,\ 0.168,\ 0.092).\end{aligned}$$

3 Eliminate G: $\tau_2(I) = \sum_g \tau_1(I, g)P(l^0 \mid g)$:

$$\tau_2(i^0) = 0.20(0.1) + 0.34(0.4) + 0.46(0.99) = 0.020 + 0.136 + 0.4554 = 0.6114,$$
$$\tau_2(i^1) = 0.74(0.1) + 0.168(0.4) + 0.092(0.99) = 0.074 + 0.0672 + 0.09108 = 0.23228 \approx 0.2323.$$

4 Multiply by $P(I) = (0.7, 0.3)$ and normalise:

$$P(i^0, l^0) = 0.7(0.6114) = 0.42798, \qquad P(i^1, l^0) = 0.3(0.23228) = 0.06968,$$
$$P(l^0) = 0.42798 + 0.06968 = 0.4977, \qquad P(i^1 \mid l^0) = \frac{0.06968}{0.4977} = 0.140,$$

down from the prior 0.30, the same value Session 6 found by enumerating all $2 \cdot 2 \cdot 3 \cdot 2 \cdot 2 = 48$ joint entries. The largest table VE built is the product $\psi(D, I, G) = P(D)P(G \mid I, D)$ in step 2, with $2 \cdot 2 \cdot 3 = 12$ entries.

PYTHON LAB · s07_examples.py · ex_student_query()

```
tau['I','G'] = [[0.2,0.34,0.46],[0.74,0.168,0.092]]   tau_2(i0) = 0.61140,
tau_2(i1) = 0.23228
P(I, L=weak) = (0.42798, 0.06968)   P(L=weak) = 0.4977   P(I=high | L=weak)
= 0.1400
```

PART I · EXACT INFERENCE

7.3 Elimination Order Matters: Treewidth

Correctness holds for *any* order, but cost does not. Each elimination step's cost is exponential in the number of variables in the largest factor *created*, since that factor's table has one entry per joint assignment of its scope. Choosing a bad order can create huge intermediate factors even when a good order would not.

Definition 7.4 (Induced graph and treewidth)**.** Given an elimination order σ, the *induced graph* adds an edge between any two variables that ever appear together in the same factor during elimination (this includes "fill-in" edges created when eliminating a variable connects all of its remaining neighbors). The *induced width* of σ is the size of the largest clique in the induced graph, minus one. The *treewidth* of the graph is the minimum induced width over all elimination orders.

For a Bayesian network the relevant undirected graph is the *moral graph* (connect co-parents, drop directions), because each CPT $P(x_i \mid \mathrm{pa}(x_i))$ is a factor whose scope $\{X_i\} \cup \mathrm{pa}(X_i)$ must be a clique.

A useful fact (Koller & Friedman 2009, Ch. 9): every product ψ formed by VE has as scope a clique of the induced graph, namely the eliminated variable Z together with its current neighbours, made pairwise adjacent by the fill-in step; and the induced graph is *chordal*, since the first-eliminated vertex of any long cycle connects its two cycle neighbours.

Proposition 7.2 (Complexity of VE)**.** *Variable elimination with order σ of induced width w costs $O(n \cdot d^{w+1})$ time, where d is the largest variable domain size and n the number of variables.*

Proof. Each ψ has a clique of the induced graph as scope, hence size at most $w+1$, hence at most d^{w+1} entries. Computing ψ costs one multiplication per entry per factor involved, and summing out Z costs $O(d^{w+1})$ additions. Every factor (initial or intermediate) is multiplied into exactly one ψ and then discarded; there are at most n initial CPTs and at most n intermediate factors. Summing over the $\le n$ elimination steps gives $O(n\, d^{w+1})$ arithmetic operations and the same order of memory. □

Figure 7.3 shows how sharply cost grows in w: this is the price of a bad ordering, and it is entirely avoidable structure — unlike the raw 2^n we started with in Session 5, which had no escape at all. Finding an order of minimum width is itself NP-hard, so in practice one uses greedy heuristics (Table 7.1).

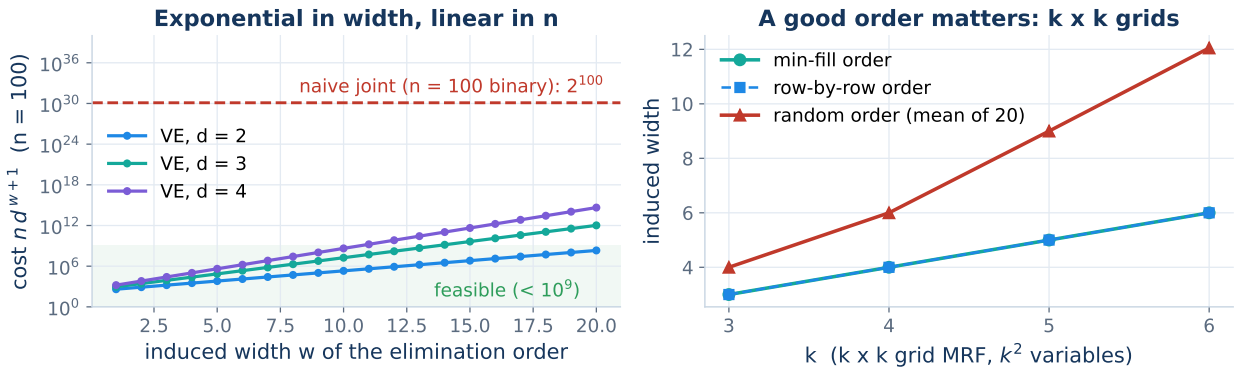


Figure 7.3. Left: the cost bound $n\,d^{w+1}$ of Proposition 7.2 for $n = 100$ variables with $d = 2, 3, 4$ values, against the 2^{100} entries of the naive joint table: exponential in the induced width w, but only linear in n for fixed w, so sparse, tree-like graphs (small w) stay tractable at any n. Right: induced widths on $k \times k$ grids, $k = 3, 4, 5, 6$. Min-fill and row-by-row orders attain the treewidth k, while uniformly random orders average $4.0,\ 6.0,\ 9.0,\ 12.1$ (mean over 20 random permutations per k; `s07_examples.py`, `ex_min_fill_grid()`).

INTUITION

A good heuristic ("min-fill"): at each step, eliminate whichever remaining variable would add the fewest fill-in edges. Intuitively, eliminate leaves and near-leaves of the graph first, and save densely-connected "hub" variables for last — eliminating a hub early forces you to connect all of its many neighbors to each other, ballooning the factor size.

Table 7.1. Greedy elimination-order heuristics and the cost of exact inference for common structures (binary or d-ary variables, n nodes).

Heuristic / structure	Rule or treewidth	Consequence
Min-degree / min-fill	fewest neighbours / fewest fill-in edges	cheap greedy orders; min-fill usually best
Chain / HMM	$w = 1$	$O(nd^2)$: forward algorithm
Tree, polytree	$w = 1$; polytree $w \le$ max in-degree	$O(nd^2)$; $O(nd^{k+1})$ with k parents
$k \times k$ grid	$w = k$	$O(k^2 d^{k+1})$: fine for $k \approx 20$ binary
Complete graph	$w = n - 1$	$O(d^n)$: no better than brute force

PART II

Belief Propagation and Message Passing

Reuse the partial sums: all marginals for the price of two

PART II · BELIEF PROPAGATION

7.4 From Variable Elimination to Messages on Trees

To get *every* marginal $P(x_i \mid e)$, running VE n times recomputes the same intermediate factors again and again. Belief propagation (BP) stores them as *messages* on the edges and reuses them.

Pairwise tree models. Let $T = (V, \mathcal{E})$ be a tree and

$$\tilde{P}(\mathbf{x}) = \prod_{i \in V} \phi_i(x_i) \prod_{(i,j) \in \mathcal{E}} \psi_{ij}(x_i, x_j). \tag{7.3}$$

A tree-structured Bayesian network has this form with $\phi_{\text{root}} = P(x_{\text{root}})$, $\psi_{ij}(x_i, x_j) = P(x_j \mid x_i)$ for each parent i of j, and the evidence absorbed as indicator unary factors $\phi_j(x_j) = \mathbb{1}[x_j = e_j]$. Then $\tilde{P}(\mathbf{x}) = P(\mathbf{x}, e)$ and the normaliser is $P(e)$.

Definition 7.5 (Sum-product messages)**.** For neighbours i, j of the tree, with $N(i)$ the set of neighbours of i, the message from i to j is the function of x_j

$$m_{i \to j}(x_j) = \sum_{x_i} \phi_i(x_i)\, \psi_{ij}(x_i, x_j) \prod_{k \in N(i) \setminus \{j\}} m_{k \to i}(x_i), \tag{7.4}$$

and the *belief* of node i is $b_i(x_i) = \phi_i(x_i) \prod_{k \in N(i)} m_{k \to i}(x_i)$.

A message can be sent once all the *other* incoming messages of its sender have arrived, so leaves start. Removing the edge (i, j) splits the tree into two parts; write $T_{i\to j}$ for the part containing i.

Theorem 7.2 (BP is exact on trees)**.** *On a tree, every message satisfies*

$$m_{i\to j}(x_j) = \sum_{\mathbf{x}_{T_{i\to j}}} \psi_{ij}(x_i, x_j) \prod_{k\in T_{i\to j}} \phi_k(x_k) \prod_{(k,l)\in\mathcal{E}(T_{i\to j})} \psi_{kl}(x_k, x_l), \tag{7.5}$$

and consequently $b_i(x_i) = \sum_{\mathbf{x}_{V\setminus i}} \tilde{P}(\mathbf{x})$*, i.e.* $b_i(x_i)/\sum_{x_i'} b_i(x_i') = P(x_i \mid e)$ *and* $\sum_{x_i} b_i(x_i) = P(e)$ *for every* i*.*

Proof. Induction on the number of nodes of $T_{i\to j}$. If $|T_{i\to j}| = 1$, i is a leaf, the product over $N(i) \setminus \{j\}$ is empty and (7.4) coincides with (7.5). Otherwise $T_{i\to j}$ consists of i and the disjoint subtrees $T_{k\to i}$, $k \in N(i) \setminus \{j\}$, each strictly smaller. The summand of (7.5) factorizes as $\psi_{ij}(x_i, x_j)\phi_i(x_i) \prod_k [\text{terms of } T_{k\to i} \text{ and } \psi_{ki}]$, and the terms of different subtrees share no variables except x_i. By the distributive law (7.2) the sum over each subtree's variables can be pushed inside, and by the induction hypothesis each such inner sum equals $m_{k\to i}(x_i)$. What remains is the sum over x_i in (7.4). For the belief, $V \setminus \{i\}$ is the disjoint union of the $T_{k\to i}$, $k \in N(i)$, and the same argument gives $\sum_{\mathbf{x}_{V\setminus i}} \tilde{P}(\mathbf{x}) = \phi_i(x_i) \prod_k m_{k\to i}(x_i) = b_i(x_i)$. □

Two-pass schedule. 1 *Collect*: pick a root; in post-order every non-root node sends its message to its parent. 2 *Distribute*: in pre-order every node sends messages to its children. 3 Output $P(x_i \mid e) = b_i / \sum b_i$ for every i. Exactly $2(n-1)$ messages are computed, each in $O(d^2)$, so *all* n marginals cost $O(nd^2)$ — the price of two VE runs. The collect pass alone *is* VE with a leaves-first order and the root as query.

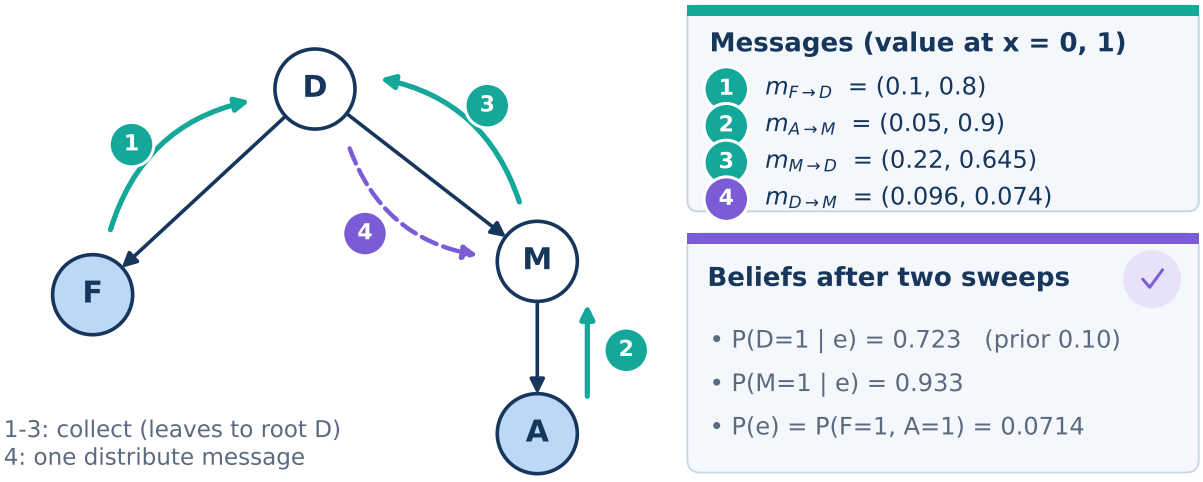


Figure 7.4. Sum-product on a hypothetical diagnostic tree with evidence $F = 1$, $A = 1$. Messages 1–3 form the collect pass towards the root D; message 4 is one of the distribute messages. Each message is a vector over the receiver's two values.

Worked Example 7.2 (Message passing by hand)**.** Hypothetical diagnostic tree (Fig. 7.4): disease D with $P(D = 1) = 0.1$; fever F with $P(F = 1 \mid D = 1) = 0.8$, $P(F = 1 \mid D = 0) = 0.1$; marker M with $P(M = 1 \mid D = 1) = 0.7$, $P(M = 1 \mid D = 0) = 0.2$; lab alarm A with $P(A = 1 \mid M = 1) = 0.9$, $P(A = 1 \mid M = 0) = 0.05$. Evidence: $F = 1$, $A = 1$. Find $P(D = 1 \mid e)$ and $P(M = 1 \mid e)$.

Solution. In the pairwise form (7.3), $\phi_D(d) = P(d)$, $\phi_F(f) = \mathbb{1}[f = 1]$, $\phi_A(a) = \mathbb{1}[a = 1]$, $\phi_M \equiv 1$, and each edge carries the child's CPT. Every message is written as the vector of its values at $(0, 1)$ of the receiving variable; complements are $P(D = 0) = 0.9$, $P(M = 0 \mid D = 0) = 0.8$, $P(M = 0 \mid D = 1) = 0.3$.

1 2 Leaves: $m_{F\to D}(d) = \sum_f \mathbb{1}[f = 1]P(f \mid d) = P(F = 1 \mid d) = (0.1, 0.8)$ and $m_{A\to M}(m) = P(A = 1 \mid m) = (0.05, 0.9)$.

3 $m_{M\to D}(d) = \sum_m P(m \mid d)\, m_{A\to M}(m) = (0.8 \cdot 0.05 + 0.2 \cdot 0.9,\ 0.3 \cdot 0.05 + 0.7 \cdot 0.9) = (0.04 + 0.18,\ 0.015 + 0.63) = (0.22,\ 0.645)$.

Belief at the root: $b_D(d) = P(d)\, m_{F\to D}(d)\, m_{M\to D}(d) = (0.9 \cdot 0.1 \cdot 0.22,\ 0.1 \cdot 0.8 \cdot 0.645) = (0.0198,\ 0.0516)$, so $P(e) = 0.0198 + 0.0516 = 0.0714$ and $P(D = 1 \mid e) = 0.0516/0.0714 = 0.723$.

4 Distribute: $m_{D\to M}(m) = \sum_d P(d)\, m_{F\to D}(d)\, P(m \mid d) = (0.9 \cdot 0.1 \cdot 0.8 + 0.1 \cdot 0.8 \cdot 0.3,\ 0.9 \cdot 0.1 \cdot 0.2 + 0.1 \cdot 0.8 \cdot 0.7) = (0.072 + 0.024,\ 0.018 + 0.056) = (0.096,\ 0.074)$, and $b_M(m) = m_{D\to M}(m)\, m_{A\to M}(m) = (0.096 \cdot 0.05,\ 0.074 \cdot 0.9) = (0.0048,\ 0.0666)$, giving $P(M = 1 \mid e) = 0.0666/0.0714 = 0.933$; note that b_M also sums to $P(e) = 0.0714$, as Theorem 7.2 predicts.

For comparison, the fever alone gives $P(D = 1 \mid F = 1) = \frac{0.1 \cdot 0.8}{0.1 \cdot 0.8 + 0.9 \cdot 0.1} = \frac{0.08}{0.17} = 0.471$; the lab alarm, two edges away, raises it to 0.723.

PYTHON LAB · s07_examples.py · ex_bp_tree()

```
m_M->D = [0.22 0.645] m_D->M = [0.096 0.074] P(F=1,A=1) = 0.0714 P(D=1|e) =
0.7227 P(M=1|e) = 0.9328
brute force:  0.7227 0.9328   P(D, F=1) = [0.09 0.08]; P(D=1 | F=1) only =
0.4706
```

Pearl's (1988) polytree algorithm (π- and λ-messages) is the same idea on directed trees.

PART II · BELIEF PROPAGATION

7.5 Chains in Time: HMM Filtering and Smoothing

A hidden Markov model (HMM) has hidden states $z_t \in \{1, \dots, K\}$, observations x_t, initial distribution π, transition matrix $A_{kl} = P(z_t = l \mid z_{t-1} = k)$ and emission probabilities $B_k(x) = P(x_t = x \mid z_t = k)$. Its graph is a chain, so its treewidth is 1.

Proposition 7.3 (Forward algorithm = VE in time order)**.** *Let $\alpha_t(k) = P(z_t = k, x_{1:t})$. Then $\alpha_1(k) = \pi_k B_k(x_1)$ and*

$$\alpha_t(l) = B_l(x_t) \sum_k \alpha_{t-1}(k) A_{kl}, \qquad P(x_{1:T}) = \sum_k \alpha_T(k), \qquad P(z_t \mid x_{1:t}) = \frac{\alpha_t(z_t)}{\sum_k \alpha_t(k)}. \tag{7.6}$$

The total cost is $O(TK^2)$.

Proof. $P(z_{1:t}, x_{1:t}) = \pi_{z_1} B_{z_1}(x_1) \prod_{s=2}^{t} A_{z_{s-1} z_s} B_{z_s}(x_s)$. Eliminating $z_1, z_2, \dots, z_{t-1}$ in this order, the only factors containing z_{t-1} at its turn are the accumulated factor $\alpha_{t-1}(z_{t-1})$ and $A_{z_{t-1} z_t}$; summing z_{t-1} out and multiplying by the untouched $B_{z_t}(x_t)$ gives (7.6). Each step is a $K \times K$ matrix–vector product. □

The backward messages $\beta_t(k) = P(x_{t+1:T} \mid z_t = k)$ obey $\beta_t(k) = \sum_l A_{kl} B_l(x_{t+1}) \beta_{t+1}(l)$, and the

smoothing marginals are $P(z_t \mid x_{1:T}) \propto \alpha_t \beta_t$: forward–backward is exactly the two-pass BP schedule on the chain, with α and β the left and right messages. In practice α_t is normalised at each step to avoid underflow; the product of the normalising constants is $P(x_{1:T})$.

Worked Example 7.3 (Monitoring an API)**.** An ops agent models a web API whose hidden mode is Up (U) or Degraded (D) (hypothetical numbers). All vectors are ordered (U, D): $\pi = (0.8, 0.2)$, $A = \left(\begin{smallmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{smallmatrix}\right)$ (row = current mode, column = next mode); emissions over {ok, slow, error}: $B_U = (0.70, 0.25, 0.05)$, $B_D = (0.20, 0.40, 0.40)$. Observed: ok, slow, slow, error, error. Filter and smooth.

Solution. We run (7.6) in normalised form: write $\hat{\alpha}_t = P(z_t \mid x_{1:t})$, predict $\bar{p}_t = \hat{\alpha}_{t-1} A$ (row vector times matrix), multiply by the emission column $B(x_t) = (B_U(x_t), B_D(x_t))$, and divide by the normaliser $c_t = P(x_t \mid x_{1:t-1})$.

$t = 1$ (ok): $\alpha_1 = (0.8 \cdot 0.70,\ 0.2 \cdot 0.20) = (0.56,\ 0.04)$, $c_1 = 0.60$, $\hat{\alpha}_1 = (0.9333,\ 0.0667)$.

$t = 2$ (slow): $\bar{p}_2 = (0.9333 \cdot 0.9 + 0.0667 \cdot 0.3,\ 0.9333 \cdot 0.1 + 0.0667 \cdot 0.7) = (0.86,\ 0.14)$; times $B(\text{slow}) = (0.25, 0.40)$ gives $(0.215,\ 0.056)$, $c_2 = 0.271$, so $P(z_2 = D \mid x_{1:2}) = 0.056/0.271 = 0.2066$.

The remaining steps are identical and are listed in Table 7.2: the filtering probabilities of Degraded are 0.0667, 0.2066, 0.3159, 0.7653, 0.9103. The likelihood is the product of the normalisers, $P(x_{1:5}) = c_1 c_2 c_3 c_4 c_5 = 0.6 \cdot 0.271 \cdot 0.2836 \cdot 0.15134 \cdot 0.24571 = 0.001715$ (log-likelihood -6.369), matching the brute-force sum over all 2^5 paths.

Smoothing needs the backward messages $\beta_t(k) = P(x_{t+1:T} \mid z_t = k)$, $\beta_5 = (1, 1)$, $\beta_t = A\,(B(x_{t+1}) \odot \beta_{t+1})$ ($\odot$ = entrywise product). For example $\beta_4 = A\,(0.05,\ 0.40)^\top = (0.9 \cdot 0.05 + 0.1 \cdot 0.40,\ 0.3 \cdot 0.05 + 0.7 \cdot 0.40) = (0.085,\ 0.295)$. Then $P(z_t \mid x_{1:5}) \propto \hat{\alpha}_t \odot \beta_t$; at $t = 3$: $(0.6841 \cdot 0.015625,\ 0.3159 \cdot 0.083875) = (0.01069,\ 0.02650)$, so $P(z_3 = D \mid x_{1:5}) = 0.02650/0.03719 = 0.7126$. Smoothing revises the past with the future errors: $P(z_t = D \mid x_{1:5}) = 0.1731, 0.4831, 0.7126, 0.9188, 0.9103$. Finally, generic VE on the factor list $\{\pi, B(x_1), A, B(x_2), A, B(x_3)\}$ with order (Z_1, Z_2) gives the same $P(Z_3 \mid x_{1:3}) = (0.6841,\ 0.3159)$ as the forward filter.

PYTHON LAB · `s07_examples.py` · `ex_hmm_filtering_api()`, `ex_chain_forward_equals_ve()`

```
t=3 P(Degraded | x_1:t) = 0.3159, P(Degraded | x_1:T) = 0.7126; P(x_1:T) =
0.001715 (brute force 0.001715)
product of the c_t = 0.6 * 0.271 * 0.2836 * 0.15134 * 0.24571 = 0.001715
VE P(Z3 | x1:3) = [0.6841 0.3159]
```

Table 7.2. Forward–backward for the API monitor of Worked Example 7.3, step by step: probability of Degraded under the predicted distribution $\bar{p}_t = \hat{\alpha}_{t-1} A$ (for $t = 1$ the prior π; $\bar{p}_t(U) = 1 - \bar{p}_t(D)$), unnormalised update $\bar{p}_t \odot B(x_t)$, normaliser c_t, filtered and smoothed probability of Degraded, and the unnormalised backward message β_t. Intermediate values are carried at full precision and rounded for display (`ex_hmm_filtering_api()`).

t	obs	$\bar{p}_t(D)$	$\bar{p}_t \odot B(x_t)$	c_t	filter D	$\beta_t(U)$	$\beta_t(D)$	smooth D
1	ok	0.2000	(0.56000, 0.04000)	0.60000	0.0667	0.0025322	0.0074192	0.1731
2	slow	0.1400	(0.21500, 0.05600)	0.27100	0.2066	0.0068706	0.024657	0.4831
3	slow	0.2240	(0.19400, 0.08959)	0.28360	0.3159	0.015625	0.083875	0.7126
4	error	0.2896	(0.03552, 0.11582)	0.15134	0.7653	0.085	0.295	0.9188
5	error	0.5592	(0.02204, 0.22367)	0.24571	0.9103	1	1	0.9103

PART II · BELIEF PROPAGATION

7.6 Clique Trees (Junction Trees)

BP is exact on trees. For a general graph we first *make* a tree — not of variables but of clusters of variables.

Definition 7.6 (Clique tree)**.** A *clique tree* for a set of factors is a tree whose nodes are clusters $C_1, \dots, C_k \subseteq \mathbf{X}$ such that (a) every factor is assigned to a cluster containing its scope, and (b) the *running intersection property* holds: if $X \in C_i \cap C_j$ then X belongs to every cluster on the path between C_i and C_j. The *separator* of an edge is $S_{ij} = C_i \cap C_j$.

Construction. (1) Moralize; (2) triangulate by symbolic VE (add the fill-in edges); (3) list the maximal cliques; (4) join them by a maximum-weight spanning tree with weights $|C_i \cap C_j|$, which satisfies running intersection. Messages are BP messages between clusters,

$$\delta_{i \to j}(S_{ij}) = \sum_{C_i \setminus S_{ij}} \beta_i^0(C_i) \prod_{k \in N(i) \setminus \{j\}} \delta_{k \to i}(S_{ki}),$$

where β_i^0 is the product of the factors assigned to C_i. After a collect and a distribute pass every cluster belief $\beta_i = \beta_i^0 \prod_k \delta_{k \to i}$ equals $\tilde{P}(C_i) = P(C_i, e)$, and neighbouring beliefs agree on their separators (*calibration*). The proof is that of Theorem 7.2 applied to the tree of clusters, with each cluster playing the role of a node: running intersection guarantees that a variable shared by two clusters lies in every separator between them, so it is never summed out before both sides have been combined (Koller & Friedman 2009, Ch. 10). The cost $O(k\, d^{\max_i |C_i|})$ is the VE price, paid once for all cluster marginals.

Worked Example 7.4 (Calibrating the student clique tree)**.** The moral student graph (edges D–I, D–G, I–G, I–S, G–L) is already chordal; its maximal cliques $C_1 = \{I, S\}$, $C_2 = \{D, I, G\}$, $C_3 = \{G, L\}$ form a chain $C_1 - C_2 - C_3$ with separators $S_{12} = \{I\}$, $S_{23} = \{G\}$. With the CPTs and value labels of Worked Example 7.1 and evidence l^0, assign $\beta_1^0 = P(I)P(S \mid I)$, $\beta_2^0 = P(D)P(G \mid I, D)$ and $\beta_3^0 = P(l^0 \mid G)$. Vectors over I are ordered (i^0, i^1).
Collect towards the root C_2: $\delta_{1 \to 2}(I) = \sum_s P(I)P(s \mid I) = P(I) = (0.7,\, 0.3)$ and $\delta_{3 \to 2}(G) = P(l^0 \mid G) = (0.1,\, 0.4,\, 0.99)$. The root belief $\beta_2 = \beta_2^0\, \delta_{1 \to 2}\, \delta_{3 \to 2} = P(D)P(G \mid I, D)P(I)P(l^0 \mid G)$, summed over D, G, is $P(I)\tau_2(I) = (0.42798,\, 0.06968)$ with τ_2 from Worked Example 7.1.
Distribute back to C_1: $\delta_{2 \to 1}(I) = \sum_{d,g} P(d)P(g \mid I, d)\, \delta_{3 \to 2}(g) = \sum_g \tau_1(I, g)P(l^0 \mid g) = (0.6114,\, 0.2323)$ — exactly the factor τ_2 of Worked Example 7.1. The belief $\beta_1(I, S) = P(I)P(S \mid I)\, \delta_{2 \to 1}(I)$ summed over S is again $(0.7 \cdot 0.6114,\ 0.3 \cdot 0.2323) = (0.42798,\, 0.06968)$: both clusters containing I report $P(i^1 \mid l^0) = 0.06968 / 0.4977 = 0.14$ (calibration). Summing β_1 over I instead gives, at no extra cost,

$$P(s^1, l^0) = 0.7 \cdot 0.05 \cdot 0.6114 + 0.3 \cdot 0.8 \cdot 0.23228 = 0.02140 + 0.05575 = 0.07715,$$
$$P(s^1 \mid l^0) = \frac{0.07715}{0.4977} = 0.155.$$

PYTHON LAB · s07_examples.py · ex_junction_tree_student()

```
separator messages:  delta_1->2(I) = [0.7 0.3], delta_2->1(I) = [0.6114
0.2323]
P(I | L=weak) from clique {D,I,G}:  [0.86 0.14] ; from clique {I,S}:  [0.86
```

```
0.14]
P(S, L=weak) = [0.4205 0.0771] (sum = 0.4977)   P(S=high | L=weak) = 0.1550
```

PART II · BELIEF PROPAGATION

7.7 Loopy Belief Propagation

On a graph *with* cycles we can still iterate (7.4) from uniform messages until they stop changing, usually with *damping* $m \leftarrow (1-\lambda)m^{\text{new}} + \lambda m^{\text{old}}$. This *loopy BP* has no general guarantee: it may oscillate, and when it converges its beliefs are approximations. Its fixed points are stationary points of the Bethe free energy (Koller & Friedman 2009, Ch. 11); it works remarkably well when couplings are weak or cycles long (turbo and LDPC decoding). Session 8 runs it on factor graphs; Session 11 develops the alternatives (sampling and variational inference).

A simulation. Take a 3×3 grid of spins $x_i \in \{-1, +1\}$ with the Ising model $\tilde{P}(\mathbf{x}) = \exp\big(\sum_i h_i x_i + J \sum_{(i,j)\in\mathcal{E}} x_i x_j\big)$, i.e. unary factors $\phi_i(x_i) = e^{h_i x_i}$ and pairwise factors $\psi_{ij} = \begin{pmatrix} e^{J} & e^{-J} \\ e^{-J} & e^{J} \end{pmatrix}$. The fields are drawn once as $h_i = 0.4\, z_i$, $z_i \sim N(0,1)$ i.i.d. (generator seed 0), and kept fixed while the coupling J varies. Loopy BP starts from uniform messages, uses damping $\lambda = 0.5$ and stops when no message changes by more than 10^{-10} (at most 2000 sweeps); its marginals are compared with the exact ones obtained by enumerating all $2^9 = 512$ configurations. The error $\max_i |P_{\text{BP}}(x_i{=}{+1}) - P(x_i{=}{+1})|$ is 0 for $J = 0$ (no loops in effect), negligible for weak couplings and about 0.14–0.18 for strong ones; the same code on a 9-node chain ($J = 1$), a tree, is exact to machine precision:

PYTHON LAB · s07_examples.py · ex_loopy_bp()

```
max |P_BP - P_exact|:  J=0.0:  0.0000, J=0.2:  0.0008, J=0.4:  0.0198, J=0.6:
0.1424, J=0.8:  0.1845, J=1.0:  0.1693
chain of 9 nodes, J=1.0:  max error = 4.44e-16 (BP exact on trees)
```

Loopy BP double-counts evidence that travels around cycles, so its beliefs tend to be overconfident.

PART III

MAP Inference: From Sums to Maxima

Max-product, Viterbi decoding, and why the best whole is not the best parts

PART III · MAP INFERENCE

7.8 MAP and Max-Product

The MAP (MPE) query asks for $\mathbf{x}^\star = \arg\max_{\mathbf{x}} P(\mathbf{x} \mid e) = \arg\max_{\mathbf{x}} \tilde{P}(\mathbf{x})$. The key observation is that the distributive law also holds for maxima of non-negative numbers:

$$\max_{z} \phi_1(\mathbf{y}_1)\, \phi_2(\mathbf{y}_2, z) = \phi_1(\mathbf{y}_1) \max_{z} \phi_2(\mathbf{y}_2, z) \qquad (\phi_1 \geq 0). \tag{7.7}$$

Proposition 7.4 (Max-product variable elimination)**.** *Replace every sum-out in Algorithm 7.1 by a max-out* $\tau(\cdot) = \max_Z \psi(\cdot, Z)$ *and keep the arg-max tables. The final factor equals* $\max_{\mathbf{w}} \tilde{P}(y, \mathbf{w})$*, and tracing the stored arg-maxes backwards in reverse elimination order recovers an MAP assignment. The cost is the same* $O(n\, d^{w+1})$*.*

Proof. Correctness of the value is the proof of Proposition 7.1 with (7.7) in place of (7.2). For the assignment: the last-eliminated variable's value is the arg-max of the final factor; given the values of all later-eliminated variables, the stored table $\arg\max_Z \psi$ for an earlier variable Z returns a value attaining the maximum of ψ with the other arguments fixed, so the product of factors at the chosen assignment attains the maximum value computed. □

In logarithms max-product becomes *max-sum*, and with negative logs *min-sum* (energy minimisation); all are one algorithm over different *semirings* (Table 7.3).

Table 7.3. One algorithm, four semirings. VE, BP and junction trees only use the two operations and the distributive law.

Version	"Add"	"Multiply"	Computes
Sum-product	$+$	$\times$	marginals, $P(e)$
Max-product	max	$\times$	MAP value and assignment
Max-sum	max	$+$ (log)	log-MAP; numerically stable
Min-sum	min	$+$ ($-$ log)	minimum energy (Session 8)

For example, given a weak letter the MPE of the student network (CPTs and labels of Worked Example 7.1) is (d^1, i^0, g^3, s^0) — a hard course, a low-intelligence student, a C and a low SAT score — with

$$P(\mathbf{x}^\star, l^0) = P(d^1)P(i^0)P(g^3 \mid i^0, d^1)P(s^0 \mid i^0)P(l^0 \mid g^3) = 0.4 \cdot 0.7 \cdot 0.7 \cdot 0.95 \cdot 0.99 = 0.1843,$$

and $P(\mathbf{x}^\star \mid l^0) = 0.1843/0.4977 = 0.370$ (`ex_max_product_student()`). This is Session 6's most probable configuration of the whole network, $(d^1, i^0, g^3, s^0, l^0)$ with probability 0.1843: since that configuration already contains l^0, it remains the best one once l^0 is observed.

PART III · MAP INFERENCE

7.9 Viterbi Decoding in HMMs

For an HMM, max-product VE in time order is the *Viterbi algorithm*. Let $\delta_t(l)$ be the probability of the best state path that ends in l at time t, $\max_{z_{1:t-1}} P(z_{1:t-1}, z_t{=}l, x_{1:t})$. Then

$$\delta_1(k) = \pi_k B_k(x_1), \qquad \delta_t(l) = B_l(x_t) \max_k \delta_{t-1}(k) A_{kl}, \qquad \mathrm{bp}_t(l) = \arg\max_k \delta_{t-1}(k) A_{kl}, \tag{7.8}$$

and the MAP path is read backwards: $z_T^\star = \arg\max_k \delta_T(k)$, $z_{t-1}^\star = \mathrm{bp}_t(z_t^\star)$. Cost $O(TK^2)$, against K^T paths for brute force.

Worked Example 7.5 (Viterbi for the API monitor)**.** Decode the most probable mode sequence for the API monitor of Worked Example 7.3 (same π, A, B and observations ok, slow, slow, error, error), and compare it with two other decoders: *max-marginal* decoding, which takes $\arg\max_k P(z_t = k \mid x_{1:5})$ separately at each t, and *greedy* left-to-right decoding, which commits to $g_1 = \arg\max_k \pi_k B_k(x_1)$ and then to $g_t = \arg\max_l A_{g_{t-1}l} B_l(x_t)$, never revising earlier choices.

Solution. We apply (7.8) with the emission values $B(\text{ok}) = (0.70, 0.20)$, $B(\text{slow}) = (0.25, 0.40)$, $B(\text{error}) = (0.05, 0.40)$; in each max the first candidate comes from U, the second from D, and the back-pointer $\text{bp}_t(\cdot)$ is the state giving the larger candidate (shown in brackets).

$$
\begin{aligned}
\delta_1 &= (0.8 \cdot 0.70,\ 0.2 \cdot 0.20) = (0.56,\ 0.04), \\
\delta_2(U) &= 0.25 \max(0.56 \cdot 0.9,\ 0.04 \cdot 0.3) = 0.25 \cdot 0.504 = 0.126 && [U] \\
\delta_2(D) &= 0.40 \max(0.56 \cdot 0.1,\ 0.04 \cdot 0.7) = 0.40 \cdot 0.056 = 0.0224 && [U] \\
\delta_3(U) &= 0.25 \max(0.126 \cdot 0.9,\ 0.0224 \cdot 0.3) = 0.25 \cdot 0.1134 = 0.02835 && [U] \\
\delta_3(D) &= 0.40 \max(0.126 \cdot 0.1,\ 0.0224 \cdot 0.7) = 0.40 \cdot 0.01568 = 0.006272 && [D] \\
\delta_4(U) &= 0.05 \max(0.02835 \cdot 0.9,\ 0.006272 \cdot 0.3) = 0.05 \cdot 0.025515 = 0.001276 && [U] \\
\delta_4(D) &= 0.40 \max(0.02835 \cdot 0.1,\ 0.006272 \cdot 0.7) = 0.40 \cdot 0.0043904 = 0.001756 && [D] \\
\delta_5(U) &= 0.05 \max(0.001276 \cdot 0.9,\ 0.001756 \cdot 0.3) = 0.05 \cdot 0.0011482 = 5.741 \times 10^{-5} && [U] \\
\delta_5(D) &= 0.40 \max(0.001276 \cdot 0.1,\ 0.001756 \cdot 0.7) = 0.40 \cdot 0.0012293 = 4.917 \times 10^{-4} && [D]
\end{aligned}
$$

(at $t = 5$ the products are formed from the unrounded $\delta_4 = (0.00127575, 0.00175616)$). Since $\delta_5(D) > \delta_5(U)$, $z_5^\star = D$; following $\text{bp}_5(D) = D$, $\text{bp}_4(D) = D$, $\text{bp}_3(D) = D$, $\text{bp}_2(D) = U$ backwards gives the path U, D, D, D, D, with joint probability $P(\mathbf{z}^\star, x_{1:5}) = \delta_5(D) = 4.917 \times 10^{-4}$ and posterior probability $4.917 \times 10^{-4} / 0.001715 = 0.287$ (Fig. 7.5).

The *max-marginal* decoding reads the smoothing marginals $P(z_t = D \mid x_{1:5}) = 0.1731, 0.4831, 0.7126, 0.9188, 0.9103$ of Worked Example 7.3 and gives U, U, D, D, D ($0.4831 < 0.5$ at $t = 2$): a different and less probable path, $0.56 \cdot (0.9 \cdot 0.25) \cdot (0.1 \cdot 0.40) \cdot (0.7 \cdot 0.40) \cdot (0.7 \cdot 0.40) = 3.951 \times 10^{-4}$. *Greedy* decoding starts with U ($0.56 > 0.04$) and at every later step compares $A_{Ul} B_l(x_t)$: $(0.9 \cdot 0.25,\ 0.1 \cdot 0.40) = (0.225, 0.04)$ for slow and $(0.9 \cdot 0.05,\ 0.1 \cdot 0.40) = (0.045, 0.04)$ for error, so it never switches to D and returns U, U, U, U, U with probability $0.56 \cdot 0.225^2 \cdot 0.045^2 = 5.741 \times 10^{-5}$, a factor $4.917 \times 10^{-4} / 5.741 \times 10^{-5} = 8.57$ below the Viterbi path.

PYTHON LAB · `s07_examples.py` · `ex_viterbi_api()`

```
t=3 delta = (0.02835, 0.006272) backptr = [0 1]; t=5 delta = (5.741e-05,
0.0004917) backptr = [0 1] (0 = Up, 1 = Degraded)
Viterbi path:  ['Up','Degraded','Degraded','Degraded','Degraded'] joint prob
= 0.0004917 P(path* | x) = 0.2868
max-marginal decoding:  ['Up','Up','Degraded','Degraded','Degraded']
0.0003951   greedy:  all 'Up' 5.741e-05   ratio 8.57
```

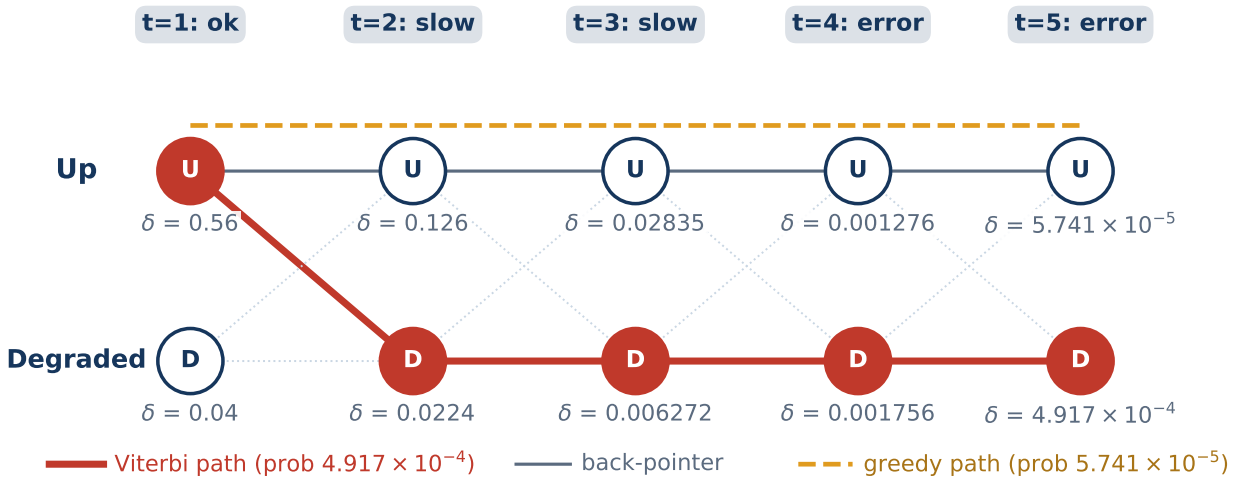


Figure 7.5. Viterbi trellis for the API monitor. Each node shows δ_t; grey edges are back-pointers, the red path is the MAP sequence obtained by backtracking from the best final state. The dashed amber line is greedy decoding, which commits early and never recovers.

PART III · MAP INFERENCE

7.10 MAP versus Marginal MAP versus Max-Marginals

Worked Example 7.6 (Three "best" answers)**.** Two binary variables $A \to B$ (no evidence) have $P(a^0) = 0.4$, $P(a^1) = 0.6$, $P(b^1 \mid a^0) = 0.9$, $P(b^1 \mid a^1) = 0.5$ (illustrative values in the spirit of Koller & Friedman 2009, Ch. 13). Find (a) the MAP assignment of (A, B), (b) the marginal MAP value of A alone, and (c) the max-marginal decoding $(\arg\max_a P(a), \arg\max_b P(b))$. *Solution.* The joint is $P(a, b) = P(a)P(b \mid a)$: $P(a^0b^0) = 0.4 \cdot 0.1 = 0.04$, $P(a^0b^1) = 0.4 \cdot 0.9 = 0.36$, $P(a^1b^0) = 0.6 \cdot 0.5 = 0.30$, $P(a^1b^1) = 0.6 \cdot 0.5 = 0.30$. (a) The largest entry is (a^0, b^1) with probability 0.36. (b) The marginal MAP for A alone sums out B: $P(a^0) = 0.04 + 0.36 = 0.4 < P(a^1) = 0.30 + 0.30 = 0.6$, so it is a^1: the best value of A *inside* the best joint configuration is not the best value of A *on its own*. (c) $P(b^1) = 0.36 + 0.30 = 0.66 > 0.5$, so decoding each variable by its own marginal gives (a^1, b^1), which has joint probability only 0.30 — not the MAP.

PYTHON LAB · s07_examples.py · ex_map_vs_marginal_map()

```
MAP (MPE) assignment:  (a0, b1) with prob 0.36   marginal MAP over A: a1
max-marginal decoding:  (a1, b1) 0.30
MAP: expected 0-1 loss = 0.64, Hamming = 0.94   max-marginal:  0-1 loss =
0.70, Hamming = 0.74
```

Marginal MAP is the hardest query: VE must sum out W *before* maximising over Y (max and sum do not commute), constraining the elimination order; it is $\mathrm{NP}^{\mathrm{PP}}$-complete in general and NP-hard even on polytrees. Which answer is "right" depends on the loss (Proposition 7.5).

PART IV

The Statistics of Inference

Gaussian conditioning, Bayes estimators, the Kalman filter and the likelihood

PART IV · MATHEMATICAL STATISTICS

7.11 Exact Inference in Linear-Gaussian Networks

Session 6 showed that a linear-Gaussian Bayesian network $X_i = \sum_{j\in\mathrm{pa}(i)} b_{ij}X_j + \varepsilon_i$, $\varepsilon_i \sim N(0, d_i)$, has a jointly Gaussian distribution with $\Sigma = (I-B)^{-1}D(I-B)^{-\top}$, where $B = (b_{ij})$ holds the edge coefficients ($b_{ij} = 0$ unless $X_j \in \mathrm{pa}(X_i)$) and $D = \mathrm{diag}(d_1, \dots, d_n)$ the noise variances; equivalently $\mathrm{Cov}(X_i, X_j) = \sum_k d_k \tau_{ik}\tau_{jk}$, where τ_{ik} is the sum over directed paths $X_k \to \cdots \to X_i$ of the products of edge coefficients ($\tau_{ii} = 1$). Exact inference then reduces to one theorem.

Theorem 7.3 (Gaussian conditioning). *Let* $\left(\begin{smallmatrix}\mathbf{x}_a\\ \mathbf{x}_b\end{smallmatrix}\right) \sim N\Big(\left(\begin{smallmatrix}\boldsymbol{\mu}_a\\ \boldsymbol{\mu}_b\end{smallmatrix}\right), \left(\begin{smallmatrix}\Sigma_{aa} & \Sigma_{ab}\\ \Sigma_{ba} & \Sigma_{bb}\end{smallmatrix}\right)\Big)$ *with* Σ_{bb} *invertible. Then*

$$\mathbf{x}_a \mid \mathbf{x}_b \sim N\Big(\boldsymbol{\mu}_a + \Sigma_{ab}\Sigma_{bb}^{-1}(\mathbf{x}_b - \boldsymbol{\mu}_b),\ \ \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}\Big). \tag{7.9}$$

Equivalently, in terms of the precision $\Lambda = \Sigma^{-1}$*:* $\mathrm{Var}(\mathbf{x}_a \mid \mathbf{x}_b) = \Lambda_{aa}^{-1}$ *and* $\mathbb{E}[\mathbf{x}_a \mid \mathbf{x}_b] = \boldsymbol{\mu}_a - \Lambda_{aa}^{-1}\Lambda_{ab}(\mathbf{x}_b - \boldsymbol{\mu}_b)$.

Proof. Put $K = \Sigma_{ab}\Sigma_{bb}^{-1}$ and $\mathbf{z} = \mathbf{x}_a - K\mathbf{x}_b$. The pair $(\mathbf{z}, \mathbf{x}_b)$ is a linear transformation of a Gaussian vector, hence jointly Gaussian, and

$$\mathrm{Cov}(\mathbf{z}, \mathbf{x}_b) = \Sigma_{ab} - K\Sigma_{bb} = 0.$$

Uncorrelated components of a jointly Gaussian vector are independent, so $\mathbf{z} \perp\!\!\!\perp \mathbf{x}_b$ and the conditional law of $\mathbf{z}$ given $\mathbf{x}_b$ is its marginal $N(\boldsymbol{\mu}_a - K\boldsymbol{\mu}_b,\ \mathrm{Var}(\mathbf{z}))$ with

$$\mathrm{Var}(\mathbf{z}) = \Sigma_{aa} - K\Sigma_{ba} - \Sigma_{ab}K^{\top} + K\Sigma_{bb}K^{\top} = \Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}.$$

Since $\mathbf{x}_a = \mathbf{z} + K\mathbf{x}_b$ and $K\mathbf{x}_b$ is fixed given $\mathbf{x}_b$, (7.9) follows. For the precision form, multiplying out shows $\Sigma^{-1} = \left(\begin{smallmatrix} S^{-1} & -S^{-1}K \\ -K^{\top}S^{-1} & \Sigma_{bb}^{-1} + K^{\top}S^{-1}K \end{smallmatrix}\right)$ with $S = \mathrm{Var}(\mathbf{z})$, so $\Lambda_{aa}^{-1} = S$ and $-\Lambda_{aa}^{-1}\Lambda_{ab} = K$. □

The posterior covariance does not depend on the observed $\mathbf{x}_b$ and is $\preceq \Sigma_{aa}$; in general models only $\mathbb{E}[\mathrm{Var}(X \mid E)] \le \mathrm{Var}(X)$ survives.

MATHEMATICAL STATISTICS · Exact inference in linear-Gaussian networks

- Inference costs $O(n^3)$ for *any* graph: Gaussians are closed under products, marginals and conditioning.
- $\mathbb{E}[\mathbf{x}_a \mid \mathbf{x}_b]$ is linear in the evidence: the population regression, with coefficient matrix K.
- Zeros of Λ_{ab} are conditional independences (Session 5): if $\Lambda_{13} = 0$ then X_3 gets coefficient 0 in $\mathbb{E}[X_1 \mid X_2, X_3]$. Sparse precision matrices make Gaussian BP fast on large graphs.

Worked Example 7.7 (A linear-Gaussian chain). $X_1 \sim N(0,1)$, $X_2 = 0.8X_1 + \varepsilon_2$ ($\mathrm{Var}\varepsilon_2 = 0.36$), $X_3 = 0.5X_2 + \varepsilon_3$ ($\mathrm{Var}\varepsilon_3 = 0.75$). Find $X_1 \mid X_3 = 2$ and $X_1 \mid X_2 = 1.5, X_3 = 2$.
Solution. All means are 0. (1) Covariance matrix (the entries of $(I-B)^{-1}D(I-B)^{-\top}$, computed directly from the structural equations, using that each ε_i is independent of the earlier variables):

$$\begin{aligned} \mathrm{Var}X_2 &= 0.8^2 \cdot 1 + 0.36 = 1, & \mathrm{Var}X_3 &= 0.5^2\mathrm{Var}X_2 + 0.75 = 1, \\ \Sigma_{12} &= \mathrm{Cov}(X_1, 0.8X_1 + \varepsilon_2) = 0.8, & \Sigma_{23} &= \mathrm{Cov}(X_2, 0.5X_2 + \varepsilon_3) = 0.5, \\ \Sigma_{13} &= \mathrm{Cov}(X_1, 0.5X_2 + \varepsilon_3) = 0.5 \cdot 0.8 = 0.4. \end{aligned}$$

(2) By (7.9) with $a = \{1\}$, $b = \{3\}$: $K = \Sigma_{13}/\Sigma_{33} = 0.4$, so $X_1 \mid X_3 = 2 \sim N(0 + 0.4 \cdot 2,\ 1 - 0.4 \cdot 0.4) = N(0.8,\ 0.84)$.
(3) With $b = \{2,3\}$: $\Sigma_{bb} = \left(\begin{smallmatrix} 1 & 0.5 \\ 0.5 & 1 \end{smallmatrix}\right)$, $\Sigma_{bb}^{-1} = \frac{1}{0.75}\left(\begin{smallmatrix} 1 & -0.5 \\ -0.5 & 1 \end{smallmatrix}\right)$, $\Sigma_{ab} = (0.8,\ 0.4)$, hence

$$K = \Sigma_{ab}\Sigma_{bb}^{-1} = \tfrac{1}{0.75}\,(0.8 - 0.2,\ -0.4 + 0.4) = (0.8,\ 0),$$
$$X_1 \mid X_2 = 1.5, X_3 = 2 \sim N\big(0.8 \cdot 1.5 + 0 \cdot 2,\ 1 - (0.8 \cdot 0.8 + 0 \cdot 0.4)\big) = N(1.2,\ 0.36).$$

The coefficient of X_3 is exactly 0 because $X_1 \perp\!\!\!\perp X_3 \mid X_2$, and indeed the precision matrix $\Lambda = \Sigma^{-1}$ has $\Lambda_{13} = 0$ while $1/\Lambda_{11} = 1/2.7778 = 0.36$.
(4) *Simulation check* (Monte Carlo, not exact): drawing $N = 400\,000$ samples of (X_1, X_2, X_3) from the structural equations (`numpy` generator `default_rng(0)`) and keeping the $n = 2181$ samples with $|X_3 - 2| < 0.05$, the kept X_1 values have mean 0.811 and variance 0.850, close to $N(0.8, 0.84)$; the small differences are Monte Carlo error and the finite window around $X_3 = 2$.

PYTHON LAB · `s07_examples.py` · `ex_gaussian_chain()`

```
X1 | X3=2 :  mean = 0.8000, var = 0.8400;   X1 | X2=1.5, X3=2 :  mean =
1.2000, var = 0.3600, coefs = [[0.8 0.  ]]
precision Lambda, first row:  [2.7778 -2.2222 0.]    posterior var from
precision:  1/Lambda_11 = 0.3600
Monte Carlo (N=400000 draws, default_rng(0); kept |X3-2|<0.05, n=2181):
mean 0.811, var 0.850
```

PART IV · MATHEMATICAL STATISTICS

7.12 Decision Theory: Which Point Estimate?

A posterior is a full distribution; a decision needs one number. Let $L(\theta, a)$ be a loss and call $a^\star(e) = \arg\min_a \mathbb{E}[L(\theta, a) \mid e]$ the *Bayes action*.

Theorem 7.4 (Bayes estimators). *Let θ have posterior $\Pi(\cdot \mid e)$.*

(a) Under squared loss $L = (\theta - a)^2$ (finite posterior variance) the Bayes action is the posterior mean $\mathbb{E}[\theta \mid e]$.

(b) Under absolute loss $L = |\theta - a|$ (finite posterior mean) the Bayes action is any posterior median.

(c) Under 0–1 loss $L = \mathbb{1}[\theta \neq a]$ with θ discrete, the Bayes action is any posterior mode, i.e. the MAP estimate.

Proof. (a) Writing $m = \mathbb{E}[\theta \mid e]$, $\mathbb{E}[(\theta - a)^2 \mid e] = \mathbb{E}[(\theta - m)^2 \mid e] + (m - a)^2$ because the cross term $2(m-a)\mathbb{E}[\theta - m \mid e]$ vanishes; the minimum is at $a = m$ and equals the posterior variance. (b) $g(a) = \mathbb{E}[|\theta - a| \mid e]$ is convex with right derivative $P(\theta \le a \mid e) - P(\theta > a \mid e) = 2F(a) - 1$ and left derivative $P(\theta < a \mid e) - P(\theta \ge a \mid e)$ (F the posterior cdf). At a median m the right derivative is ≥ 0 and the left one ≤ 0, so m is a global minimiser of the convex g. (c) $\mathbb{E}[\mathbb{1}[\theta \ne a] \mid e] = 1 - P(\theta = a \mid e)$, minimised by maximising $P(\theta = a \mid e)$. □

Proposition 7.5 (MAP versus max-marginals)**.** *For a vector* $\mathbf{x} = (x_1, \dots, x_n)$*, the MAP assignment minimises the expected* 0–1 loss on the whole vector, $\mathbb{1}[\mathbf{x} \ne \mathbf{a}]$*, whereas the max-marginal assignment* $a_i = \arg\max_{x_i} P(x_i \mid e)$ *minimises the expected* Hamming loss $\sum_i \mathbb{1}[x_i \ne a_i]$.

Proof. The first claim is Theorem 7.4(c) with $\theta = \mathbf{x}$. For the second, linearity of expectation gives $\mathbb{E}[\sum_i \mathbb{1}[x_i \ne a_i] \mid e] = \sum_i \big(1 - P(x_i = a_i \mid e)\big)$, a sum of terms each depending on one coordinate only, so it is minimised coordinate-wise. □

In Worked Example 7.6 the MAP (a^0, b^1) has expected 0–1 loss $1 - 0.36 = 0.64$ but expected Hamming loss $(1 - P(a^0)) + (1 - P(b^1)) = 0.6 + 0.34 = 0.94$; the max-marginal (a^1, b^1) has expected 0–1 loss $1 - 0.30 = 0.70$ and Hamming loss $(1 - 0.6) + (1 - 0.66) = 0.4 + 0.34 = 0.74$. Each decoder wins under its own loss. Neither is "correct" in general: per-symbol error rates (e.g. in part-of-speech tagging) are Hamming-type losses, while diagnosis needs a coherent joint explanation.

Worked Example 7.8 (Three estimates of a robot's position)**.** The corridor robot of Worked Example 7.18 (Part V; 16 cells numbered 0–15, doors at $2, 5, 11$) has, after three readings (wall, wall, door), the belief $b_3(s) = P(s_3 = s \mid o_{1:3})$

s	0	1	2	3	4	5	6	7
$b_3(s)$	0.0325	0.0325	0.2526	0.0083	0.0121	0.2158	0.0080	0.0121
s	8	9	10	11	12	13	14	15
$b_3(s)$	0.0281	0.0322	0.0325	0.2526	0.0083	0.0121	0.0281	0.0322

with two tied modes at the door cells 2 and 11 and a third peak at door 5. Treating the cell index as a number on $\{0, \dots, 15\}$ (distances measured along the corridor, ignoring the wrap-around of the ring), compute the three Bayes estimates of Theorem 7.4 and their expected losses.

Solution. Mean: $\sum_s s\, b_3(s) = 0(0.0325) + 1(0.0325) + 2(0.2526) + \cdots + 15(0.0322) = 6.5749 \approx 6.575$ (computed from the unrounded belief; the 4-decimal table gives 6.574). *Median:* the cumulative sums are 0.0325, 0.0649, 0.3175, 0.3258, 0.3379, 0.5537 for $s = 0, \dots, 5$, so the first cell with cumulative probability ≥ 0.5 is 5. *Modes:* 2 and 11 (0.2526 each); we report 2.
Squared loss. By the decomposition in the proof of Theorem 7.4(a), $\mathbb{E}[(s-a)^2] = \mathrm{Var} + (6.5749 - a)^2$ with posterior variance $\mathrm{Var} = \sum_s (s - 6.5749)^2\, b_3(s) = 18.580$: at the mean 18.580, at the median $18.580 + 1.5749^2 = 18.580 + 2.480 = 21.060$, at the mode $18.580 + 4.5749^2 = 18.580 + 20.930 = 39.510$. *Absolute loss* $\sum_s |s - a|\, b_3(s)$: 3.911 at the mean, 3.732 at the median (the minimum), 4.770 at the mode. *0–1 loss* $1 - b_3(a)$: 1.000 at the mean (6.575 is not a cell), $1 - 0.2158 = 0.784$ at the median, $1 - 0.2526 = 0.747$ at the mode (the minimum). The posterior mean lies near cell 7, which carries probability 0.012 — optimal under squared loss, yet almost certainly *not* where the robot is.

PYTHON LAB · s07_examples.py · ex_bayes_estimators()

```
posterior mean = 6.575, posterior median = 5, posterior mode (MAP) = 2;
posterior variance = 18.580
mean 6.575:  E[sq]=18.580 E[abs]=3.911 E[0-1]=1.000 | median 5:  21.060
3.732 0.784 | mode 2:  39.510 4.770 0.747
probability mass at the posterior-mean cell 7:  0.0121
```

MATHEMATICAL STATISTICS · Which point estimate?

Mean, median and mode coincide for symmetric unimodal posteriors (e.g. Gaussian) and differ for skewed or multimodal ones. The MAP of a continuous parameter is not invariant under reparametrisation (Jacobian), the median is (for monotone maps); with a flat prior MAP is the MLE of Session 10. Inference supplies the posterior; the loss chooses the summary.

PART IV · MATHEMATICAL STATISTICS

7.13 The Kalman Filter is Belief Propagation

MATHEMATICAL STATISTICS · Kalman filter = BP on a linear-Gaussian chain

Take the scalar state-space model $x_t = x_{t-1} + w_t$, $w_t \sim N(0,q)$, $y_t = x_t + v_t$, $v_t \sim N(0,r)$ — an HMM whose states are continuous and whose CPDs are linear-Gaussian. Let the filtering belief be $x_{t-1} \mid y_{1:t-1} \sim N(m,P)$.

- **Predict** = sum-product message with an integral instead of a sum: $\int N(x_{t-1};m,P)\,N(x_t;x_{t-1},q)\,dx_{t-1} = N(x_t;m,P^-)$ with $P^- = P+q$ (sum of independent Gaussians).
- **Update** = multiply by the evidence factor and normalise, i.e. Theorem 7.3 applied to (x_t,y_t) with covariance $\begin{pmatrix} P^- & P^- \\ P^- & P^-+r \end{pmatrix}$: gain $K = P^-/(P^-+r)$, mean $m + K(y_t - m)$, variance $(1-K)P^-$.

The variance recursion does not involve the data: the new filtering variance is $P_t = (1-K)P^- = \frac{(P_{t-1}+q)\,r}{P_{t-1}+q+r}$. For constant q,r it converges to the fixed point $P = \frac{(P+q)r}{P+q+r}$, i.e. $P^2+qP+rP = rP+qr$, i.e. the positive root of $P^2+qP-qr=0$, $P^\star = \frac{1}{2}\big(-q+\sqrt{q^2+4qr}\big)$. Convergence from any $P_0 \geq 0$ follows because $f(P) = \frac{(P+q)r}{P+q+r}$ has $0 < f'(P) = \frac{r^2}{(P+q+r)^2} \leq \frac{r^2}{(q+r)^2} < 1$ (for $q>0$), so f is a contraction on $[0,\infty)$.

Worked Example 7.9 (One Kalman step)**.** Model of the box above with $q=0.5$, $r=1$; current belief $x_{t-1} \mid y_{1:t-1} \sim N(m,P)$ with $m=0$, $P=1$; new measurement $y_t = 1.2$. *Predict:* $P^- = P+q = 1+0.5 = 1.5$, predicted mean $m=0$. *Update:* $K = P^-/(P^-+r) = 1.5/2.5 = 0.6$, new mean $m + K(y_t-m) = 0+0.6(1.2-0) = 0.72$, new variance $(1-K)P^- = 0.4\cdot 1.5 = 0.60$ — identical to conditioning the joint Gaussian of (x_t,y_t) with covariance $\left(\begin{smallmatrix} 1.5 & 1.5 \\ 1.5 & 2.5 \end{smallmatrix}\right)$ by (7.9): mean $\frac{1.5}{2.5}\cdot 1.2 = 0.72$, variance $1.5 - \frac{1.5^2}{2.5} = 0.60$. *Steady state:* $P^2 + 0.5P - 0.5 = 0$ gives $P^\star = \frac{1}{2}\big(-0.5+\sqrt{0.25+2}\big) = \frac{1}{2}(-0.5+1.5) = 0.5$.

PYTHON LAB · s07_examples.py · ex_kalman_step()

```
predict:  mean 0.00, var 1.50; gain K = 0.60; update:  mean 0.72, var 0.60;
via conditioning:  0.72, 0.60
steady state:  P^2 + 0.5P - 0.5 = 0, sqrt(q^2 + 4qr) = 1.5000; steady-state
variance 0.5000
```

The likelihood is an inference query. With parameters θ and hidden variables $\mathbf{h}$, $P(e;\theta) = \sum_{\mathbf{h}} P(\mathbf{h}, e;\theta)$ is the likelihood. EM (Session 10) needs the posteriors $P(\mathbf{h} \mid e;\theta)$ at every iteration; the HMM likelihood costs $O(TK^2)$ by the forward algorithm (Worked Example 7.3: $\log P(x_{1:5}) = -6.369$); and likelihood gradients are inference queries too. Indeed, $P(e) = \sum_{\mathbf{h}} \prod_i \theta_{x_i|\mathbf{u}_i}$ is a sum of products in which each term contains the CPT entry $\theta_{x|u} = P(X{=}x \mid \mathrm{pa}(X){=}u)$ at most once (exactly when that term has $X = x$, $\mathrm{pa}(X) = u$); differentiating term by term therefore gives $\partial P(e)/\partial\theta_{x|u} = P(x,u,e)/\theta_{x|u}$, and dividing by $P(e)$,

$$\frac{\partial \log P(e)}{\partial \theta_{x|u}} = \frac{P(x,u \mid e)}{\theta_{x|u}},$$

a family marginal that one calibrated clique tree delivers for every CPT at once (treating the θ's as free, unnormalised parameters).

PART IV · MATHEMATICAL STATISTICS

7.14 Worked Examples

Worked Example 7.10 (Elimination order changes cost, not the answer)**.** Consider the moralized skeleton of the student network (Session 6): edges $D{-}I$ (married parents of G), $D{-}G$, $I{-}G$, $I{-}S$, $G{-}L$. Compare eliminating in order (L, S, G, D, I) versus eliminating I first.

Solution. See Fig. 7.6. Eliminating L then S then G then D then I: L's only neighbor is G (cost $O(d^2)$, no fill-in); S's only neighbor is I (no fill-in); G's remaining neighbors are D, I, already connected (no fill-in, cost $O(d^3)$); then D has the single neighbour I, and I is last. The largest product ψ, formed when eliminating G, has scope $\{G, D, I\}$ of size 3 (the largest clique of the induced graph), so the induced width is $w = 3 - 1 = 2$. Eliminating I *first* instead: I's neighbors are D, G, S; neither D–S nor G–S is an edge yet, so eliminating I forces two new fill-in edges; the product ψ has scope $\{I, D, G, S\}$ of size 4 and the new factor τ has scope $\{D, G, S\}$, giving induced width $w = 4 - 1 = 3$. Same final answer $P(y, e)$ either way (correctness doesn't depend on order; both orders give $P(I \mid l^0) = (0.86, 0.14)$) but a strictly worse induced width from the bad order.

PYTHON LAB · s07_examples.py · ex_elim_order()

```
order [L,S,G,D,I]: induced width = 2, fill-in [];   order [I,L,S,G,D]: width
3, fill-in [(D,S), (G,S)]
```

Worked Example 7.11 (Computing a query by hand)**.** Using the student network's factors $P(D)$, $P(I)$, $P(G \mid D, I)$, $P(S \mid I)$, $P(L \mid G)$, compute $P(I \mid L = \text{weak})$ symbolically via VE with order (S, D, G) eliminated before normalizing over I.

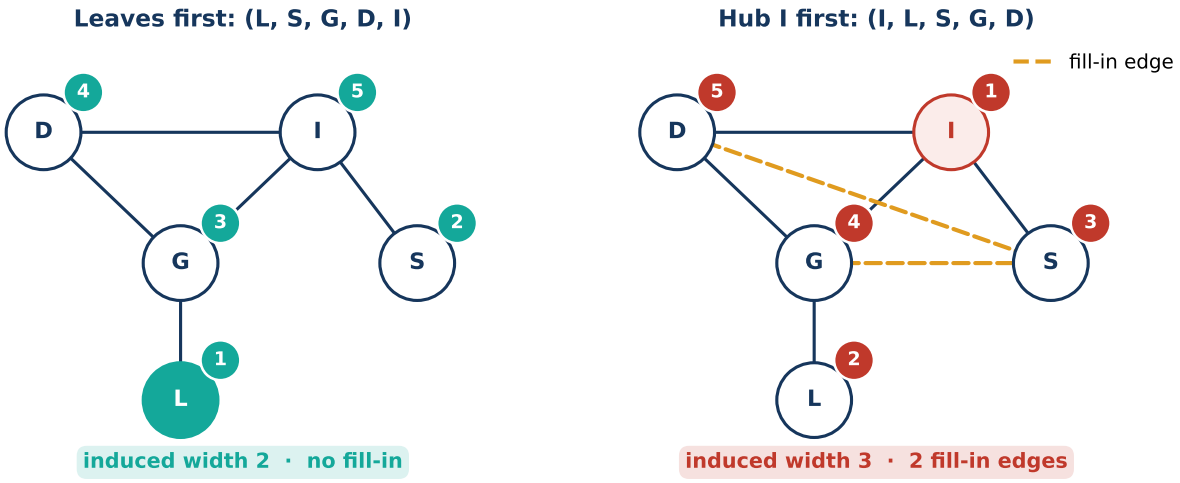


Figure 7.6. Left: eliminating leaves first needs no fill-in edges. Right: eliminating the well-connected node I first forces new (amber, dashed) edges among its neighbors. Numbers give the elimination order.

Solution. Step 1 (eliminate S): the only factor mentioning S is $P(S \mid I)$; summing over all values of S gives $\sum_s P(s \mid i) = 1$ for every i (a CPT's rows sum to one), so this factor simply vanishes — reflecting that the SAT score is a *barren* leaf: neither observed nor queried, it cannot influence the answer. Step 2 (eliminate D): multiply $P(D)$ and $P(G \mid D, I)$, then sum over D: $\tau_1(G, I) = \sum_d P(d)P(G \mid d, I)$. Step 3 (eliminate G): multiply $\tau_1(G, I)$ by the evidence-restricted factor $P(L{=}\text{weak} \mid G)$, then sum over G: $\tau_2(I) = \sum_g \tau_1(g, I)\, P(L{=}\text{weak} \mid g)$. Finally, $P(I, L{=}\text{weak}) = P(I) \cdot \tau_2(I)$, and normalizing over the two values of I gives $P(I \mid L{=}\text{weak})$ — numerically 0.14 for i^1 (Worked Example 7.1). Notice that S dropped out for free — a query never needs to touch barren variables, or variables that are d-separated from it given the evidence, which is exactly why Session 6's graphical reasoning and this session's computational reasoning are two views of the same fact.

Worked Example 7.12 (A recommendation-system factor graph)**.** A streaming platform models three variables per session: a latent taste cluster $C \in \{1, \ldots, K\}$, a genre exposure indicator X (did the system show genre g), and an observed click Y. Factors: $\phi_1(C) = P(C)$, $\phi_2(C, X) = P(X \mid C)$, $\phi_3(C, X, Y) = P(Y \mid C, X)$ (click probability depends on both taste and what was shown). To compute $P(C \mid Y = 1)$ when the exposure X is not recorded, reduce ϕ_3 to $Y = 1$ and eliminate X: $\tau(C) = \sum_x \phi_2(C, x)\phi_3(C, x, Y{=}1) = P(Y{=}1 \mid C)$; then normalize $\phi_1(C)\,\tau(C)$ over the K values of C: $P(C = c \mid Y = 1) = \phi_1(c)\tau(c) / \sum_{c'} \phi_1(c')\tau(c')$. Since X is binary, this costs $2K$ products for τ and $O(K)$ operations for the normalisation. Note that the intermediate factor $\tau(C) = P(Y{=}1 \mid C)$ is not one of the network's CPTs: VE manipulates arbitrary non-negative tables over sets of variables. This factor view — factors need not correspond to a single "child given parents" CPT — generalizes the Bayesian network picture and is exactly the representation Session 8 will formalize for undirected models.

Additional Worked Examples (Instructor Supplement)

Worked Example 7.13 (Factor product and sum-out, fully by hand)**.** The smallest possible numeric pass of variable elimination, with every factor entry written out. Let $A \to B \to C$ with $\phi_1(A,B) = P(B \mid A)$ given by $\phi_1(0,0)$=0.6, $\phi_1(0,1)$=0.4, $\phi_1(1,0)$=0.3, $\phi_1(1,1)$=0.7, and $\phi_2(B,C) = P(C \mid B)$ given by $\phi_2(0,0)$=0.9, $\phi_2(0,1)$=0.1, $\phi_2(1,0)$=0.2, $\phi_2(1,1)$=0.8. Eliminate B: form $\psi(A,B,C) = \phi_1(A,B)\phi_2(B,C)$, then $\tau(A,C) = \sum_B \psi(A,B,C)$.

$$\tau(0,0) = \phi_1(0,0)\phi_2(0,0) + \phi_1(0,1)\phi_2(1,0) = 0.6(0.9) + 0.4(0.2) = 0.54 + 0.08 = \mathbf{0.62},$$

$$\tau(0,1) = 0.6(0.1) + 0.4(0.8) = 0.06 + 0.32 = \mathbf{0.38},$$

$$\tau(1,0) = 0.3(0.9) + 0.7(0.2) = 0.27 + 0.14 = \mathbf{0.41},$$

$$\tau(1,1) = 0.3(0.1) + 0.7(0.8) = 0.03 + 0.56 = \mathbf{0.59}.$$

As a sanity check, each row sums to 1 (0.62+0.38=1, 0.41+0.59=1), confirming $\tau(A,C) = P(C \mid A)$ exactly, since eliminating the middle node of a chain by factor product and sum-out is nothing but matrix multiplication of the two conditional probability tables — the general VE algorithm doing, mechanically, what we would otherwise compute by the law of total probability.

Worked Example 7.14 (Chaining the computation to a full query)**.** Extend Worked Example 7.13 with a fourth variable D, $C \to D$, $P(D{=}1 \mid C{=}0) = 0.3$, $P(D{=}1 \mid C{=}1) = 0.7$, and prior $P(A{=}1) = 0.5$. Compute $P(D{=}1)$ by continuing variable elimination: eliminate A next, then C. Eliminating A from $\tau(A,C)$: $P(C{=}0) = \sum_a P(a)\tau(a,0) = 0.5(0.62) + 0.5(0.41) = 0.31 + 0.205 = \mathbf{0.515}$, and $P(C{=}1) = 0.5(0.38) + 0.5(0.59) = 0.19 + 0.295 = \mathbf{0.485}$ (summing to 1, as required). Finally eliminate C: $P(D{=}1) = P(C{=}0)(0.3) + P(C{=}1)(0.7) = 0.515(0.3) + 0.485(0.7) = 0.1545 + 0.3395 = \mathbf{0.494}$. Each step needed at most eight multiplications (forming $\psi(A,B,C)$, the largest table, with $2^3 = 8$ entries) and four additions; no table over all four variables ($2^4 = 16$ entries) was ever built — Proposition 7.2's cost guarantee ($w = 2$ for this order, since $\psi(A,B,C)$ has three variables), here realized concretely rather than asymptotically.

PYTHON LAB · s07_examples.py · ex_factor_product(), ex_chain_full_query()

```
tau = [[0.62, 0.38], [0.41, 0.59]]   P(C=0) = 0.515, P(C=1) = 0.485   P(D=1)
= 0.4940
```

Worked Example 7.15 (Trees always have treewidth 1)**.** Prove that *any* tree-structured model has treewidth exactly 1, regardless of how many variables $n \geq 2$ it has. (For a directed tree — every node has at most one parent — there are no v-structures, so moralization adds no edges and the moral graph is the undirected tree itself.)

Solution. Eliminate variables in a *leaf-first* order: at every step, some node of the current (shrinking) tree has at most one remaining neighbor (every tree with ≥ 2 nodes has at least one leaf). Eliminating a node with a single neighbor creates *no* fill-in edges at all (fill-in only connects *pairs* of a node's remaining neighbors, and a single neighbor has no partner to connect to), and after removal the remaining graph is still a tree (removing a leaf from a tree yields a tree). Repeating this $n - 1$ times, every elimination step operates on a factor of scope at most 2 (the eliminated variable and its one neighbor), giving induced width

$w = 1$ throughout; since any edge forces width at least 1, the treewidth is exactly 1. Since this holds for *every* n, variable elimination on a tree-structured model costs only $O(n \cdot d^2)$ — linear in the number of variables — no matter how large the tree is. This is the $w = 1$ point of the left panel of Fig. 7.3: for fixed width the cost grows only linearly in n. It is the same leaf-first principle Worked Example 7.10 used informally on the (non-tree, but still low-treewidth) student network. A simulation confirms it: on uniformly random labelled trees with $n = 10, 100, 1000$ nodes (`networkx` random trees, generator seed 3), eliminating at each step a remaining node of minimum degree gives induced width 1 and zero fill-in edges every time (`ex_tree_width_one()`).

New Analytical Examples

Worked Example 7.16 (Grids have treewidth at most k)**.** Show that the $k \times k$ grid has induced width at most k under row-by-row elimination, so VE on a $k \times k$ grid MRF costs $O(k^2 d^{k+1})$.

Solution. Eliminate row by row, left to right. Invariant: when (r, c) is eliminated its neighbours are the rest of its row, $(r, c{+}1), \dots, (r, k)$, and $(r{+}1, 1), \dots, (r{+}1, c)$ — a frontier of at most k vertices. It holds for $(1, 1)$, and eliminating (r, c) makes its frontier a clique, so $(r, c{+}1)$ inherits the frontier minus itself plus its own neighbour $(r{+}1, c{+}1)$. Hence width $\le k$; the min-fill simulation of Fig. 7.3 attains k.

Worked Example 7.17 (Forward–backward is BP on a chain)**.** Write an HMM as a pairwise chain with $\phi_t(z_t) = B_{z_t}(x_t)$ (times π_{z_1} for $t = 1$) and $\psi_{t-1,t} = A$. Show that the left-to-right message into z_t is $\alpha_{t-1}A$ and the right-to-left message is β_t.

Solution. Treat messages and ϕ_t as row vectors over the K states, write $\odot$ for the entrywise product and $B(x_t) = (B_1(x_t), \dots, B_K(x_t))$, and set $m_{0\to1} \equiv 1$, $m_{T+1\to T} \equiv 1$ (no neighbour). Since $\psi_{t-1,t}(z_{t-1}, z_t) = A_{z_{t-1} z_t}$, (7.4) reads $m_{t-1\to t}(z_t) = \sum_{z_{t-1}} (\phi_{t-1} \odot m_{t-2\to t-1})(z_{t-1}) A_{z_{t-1} z_t}$, i.e. $m_{t-1\to t} = (\phi_{t-1} \odot m_{t-2\to t-1})A$. If $\phi_{t-1} \odot m_{t-2\to t-1} = \alpha_{t-1}$ then $\phi_t \odot m_{t-1\to t} = B(x_t) \odot (\alpha_{t-1}A) = \alpha_t$ by (7.6), and $\phi_1 = \alpha_1$ starts the induction. Symmetrically, summing over the *second* index of A, $m_{t+1\to t}(z_t) = \sum_{z_{t+1}} A_{z_t z_{t+1}} B_{z_{t+1}}(x_{t+1})\, m_{t+2\to t+1}(z_{t+1})$, i.e. $m_{t+1\to t} = A\,(B(x_{t+1}) \odot m_{t+2\to t+1})$ in column-vector form, is the backward recursion with $m_{T+1\to T} \equiv 1$, and $b_t = \alpha_t \beta_t \propto P(z_t \mid x_{1:T})$.

PART V

Inference in Agents, RL and Multi-Agent Systems

The reasoning step behind perception, decoding, planning and coordination

PART V · AI CONNECTIONS

7.15 Inference as the Reasoning Step of an Agent

The rational agent of course session 1 maps percepts to actions through an internal state. Under partial observability that state is a *belief* — a posterior — and keeping it current is exactly the

inference problem of this session; the decision rule then acts on a Bayes summary of it (Theorem 7.4, Fig. 7.7).

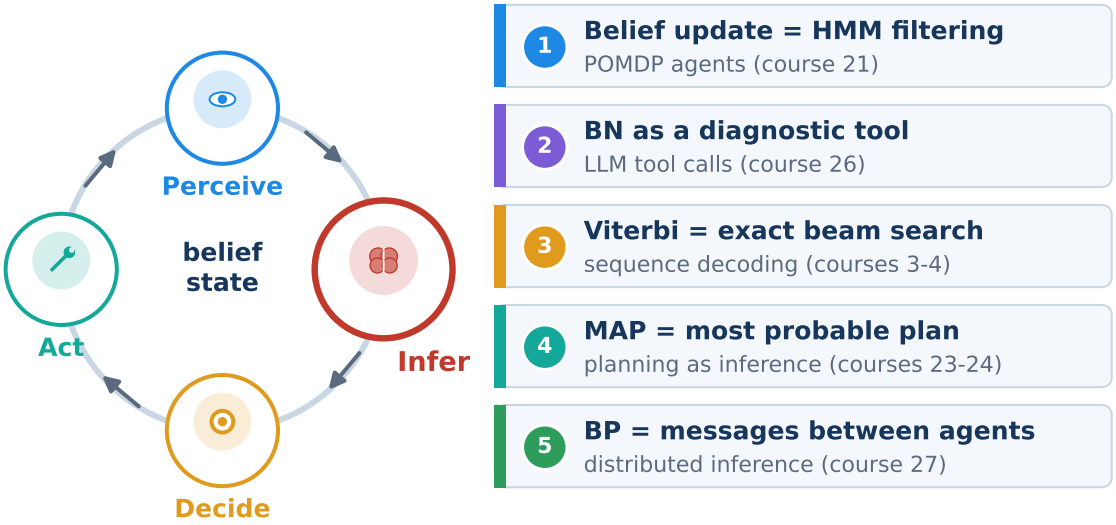


Figure 7.7. Inference sits between perception and decision in every agent. Right: five places in the course where today's algorithms reappear.

AI CONNECTION · Agentic AI

An LLM operations agent receives alerts and calls a `bn_query` tool backed by a small hypothetical Bayesian network $L \leftarrow C \rightarrow E$: root cause $C \in \{\text{DB overload, Network, Bad deploy}\}$ with prior $(0.3, 0.2, 0.5)$, latency alert $P(L=1 \mid C) = (0.9, 0.8, 0.3)$, error-rate alert $P(E=1 \mid C) = (0.2, 0.4, 0.9)$ (vectors in the order of the three causes). Given $L=1, E=0$, VE multiplies the factors $P(c)\,P(L{=}1 \mid c)\,P(E{=}0 \mid c)$:

$$(0.3 \cdot 0.9 \cdot 0.8,\ 0.2 \cdot 0.8 \cdot 0.6,\ 0.5 \cdot 0.3 \cdot 0.1) = (0.216,\ 0.096,\ 0.015), \qquad P(e) = 0.327,$$

so the posterior is $(0.216, 0.096, 0.015)/0.327 = (0.6606,\ 0.2936,\ 0.0459)$, with entropy $H = -\sum_c p_c \log_2 p_c = 1.118$ bits. Two tools could be called next, each returning positive/negative ($T = 1/0$): `db_metrics` (positive w.p. 0.95 under DB overload, 0.10 otherwise) and `ping_test` (0.90 under Network, 0.05 otherwise). The expected entropy reduction of C equals the mutual information $I(C;T) = H(T) - \sum_c P(c \mid e)\,H(T \mid C{=}c)$, with $h(p) = -p\log_2 p - (1-p)\log_2(1-p)$:

- `db_metrics`: $P(T{=}1) = \frac{0.95(0.216)+0.10(0.096+0.015)}{0.327} = \frac{0.2163}{0.327} = 0.6615$; $I = h(0.6615) - [0.6606\,h(0.95) + 0.3394\,h(0.10)] = 0.9234 - [0.6606(0.2864) + 0.3394(0.4690)] = 0.9234 - 0.3484 = 0.5750$ bits;
- `ping_test`: $P(T{=}1) = \frac{0.90(0.096)+0.05(0.216+0.015)}{0.327} = \frac{0.09795}{0.327} = 0.2995$; $I = h(0.2995) - [0.2936\,h(0.90) + 0.7064\,h(0.05)] = 0.8807 - [0.2936(0.4690) + 0.7064(0.2864)] = 0.8807 - 0.3400 = 0.5407$ bits.

Since $0.5750 > 0.5407$ the agent calls `db_metrics` first: a calibrated, auditable tool choice, obtained from a handful of further inference queries.

PYTHON LAB · s07_examples.py · ex_agent_diagnosis()

```
P(L=1, E=0) = 0.3270; posterior:  0.6606, 0.2936, 0.0459 (entropy 1.118
bits)
db_metrics:  P(positive) = 0.6615, H(T) = 0.9234, sum_c P(c|e) H(T|c) =
0.3484, I(C;T) = 0.5750 bits
ping_test :  P(positive) = 0.2995, H(T) = 0.8807, sum_c P(c|e) H(T|c) =
0.3400, I(C;T) = 0.5407 bits
```

LINK TO COURSE SESSION 26 · Agentic AI

An agent's belief over hidden task variables is its working memory; HMM/BN tools (e.g. the API monitor of Worked Example 7.3) keep it calibrated and value of information picks the next tool call.

Worked Example 7.18 (Robot localisation: the POMDP belief update)**.** A robot moves clockwise in a ring of 16 cells numbered 0–15: at each step it advances one cell w.p. 0.8, stays w.p. 0.1 and advances two cells w.p. 0.1 (cell 15 is followed by cell 0). Doors are at cells $2, 5, 11$; at each step its door sensor reads "door" w.p. 0.85 at a door cell and w.p. 0.10 at any other cell ("wall" otherwise). The belief starts uniform, $b_0(s) = 1/16$, and is updated by

$$b_t(s') \propto P(o_t \mid s') \sum_s P(s' \mid s)\, b_{t-1}(s),$$

which is the forward recursion (7.6) with $\hat{\alpha}_t = b_t$ (at $t = 1$ the motion step leaves the uniform belief unchanged). The run is *simulated* (`robot_trajectory()`, generator seed 7): the robot starts in cell 0, its true cells for $t = 1, \ldots, 12$ are $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 11, 12$ and it reads wall, wall, door, wall, wall, door, wall, wall, door, wall, door, wall (the door reading at $t = 9$, in cell 8, is a false alarm).

Solution. At $t = 1$ (wall) every wall cell gets $0.9/(13 \cdot 0.9 + 3 \cdot 0.15) = 0.9/12.15 = 0.0741$ and every door cell $0.15/12.15 = 0.0123$: one reading cannot locate the robot. After the first door ($t = 3$) the belief is split over the three doors (0.253 at cells 2 and 11, 0.216 at cell 5; full table in Worked Example 7.8). After the second door the door *spacing* disambiguates (only door 2 is followed three cells later by another door) and 0.533 sits on the true cell 5 at $t = 6$, with entropy 2.54 bits against $\log_2 16 = 4$ bits for the uniform belief (Fig. 7.8). Motion noise and the false alarm at $t = 9$ spread the belief again, and the real door at $t = 11$ brings the maximum back onto the true cell.

PYTHON LAB · s07_examples.py · ex_robot_localization()

```
t=1 (uniform prior x P(wall|s)):  wall cells 0.9/12.15 = 0.0741, door cells
0.15/12.15 = 0.0123
t= 3:  MAP cell 11 (p=0.253), 2nd 2 (p=0.253)   t= 6:  MAP cell 5 (p=0.533),
entropy 2.54 bits, true 5
t=11:  MAP cell 11 (p=0.345), 2nd 5 (p=0.124), entropy 3.18 bits, true 11
```

LINK TO COURSE SESSION 21 · Decision Making under Uncertainty

A POMDP policy conditions on the belief $b_t = P(s_t \mid o_{1:t}, a_{1:t-1})$, a sufficient statistic of the history, updated by HMM filtering with action-dependent transitions in $O(|S|^2)$ per step; continuous states use the Kalman filter (Example 7.9) or particle filters (Session 11).

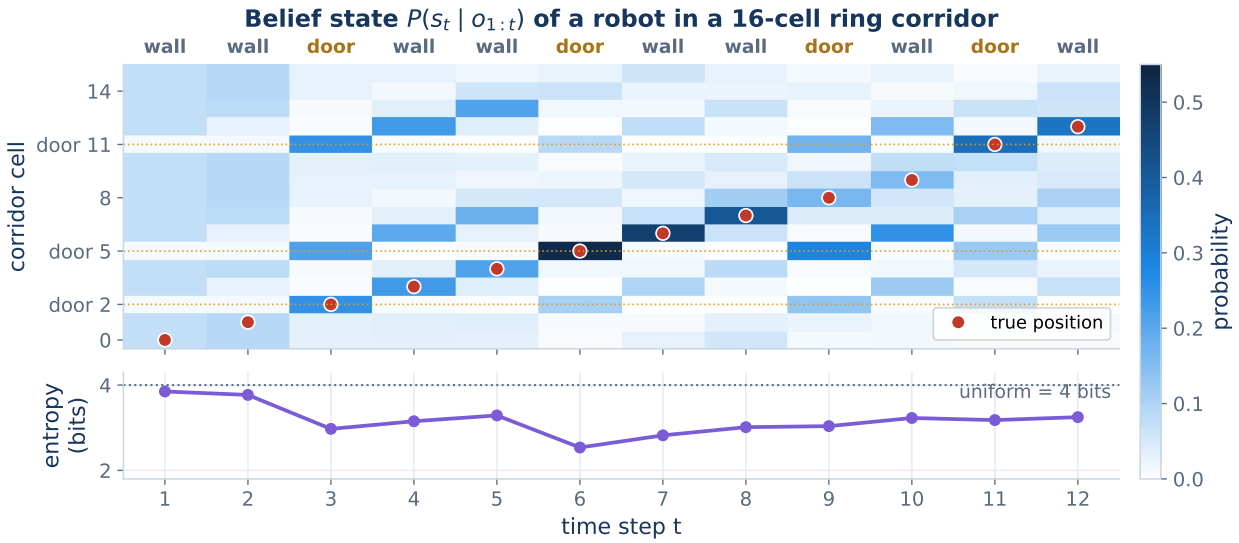


Figure 7.8. Belief state of the corridor robot (darker = more probable; red = true position). Door readings sharpen the belief, motion spreads it: the predict–update rhythm of every POMDP agent.

AI CONNECTION · Reinforcement Learning

In an MDP (course session 17) the state is its own belief; inference enters RL via partial observability, learned world models (course session 20) and *control as inference*: with $P(\mathcal{O}_t{=}1 \mid s_t, a_t) \propto e^{r(s_t,a_t)}$, backward messages are soft value functions.

AI CONNECTION · Generative Models

Greedy decoding picks the locally best token; beam search keeps the k best prefixes. For an HMM with K states, keeping the best path into each state *is* Viterbi and is exact; in Worked Example 7.5 greedy decoding never leaves Up and its path is $4.917 \times 10^{-4} / 5.741 \times 10^{-5} = 8.57$ times less probable than the MAP path. For LLMs (course sessions 3–4) the state is the whole prefix, so beam search is the practical compromise.

LINK TO COURSE SESSION 23 · Classical and Hierarchical Planning (and 24)

Planning as inference: condition on "success" and ask for the most probable plan (a MAP query). Hypothetically: two steps, each *safe* or *fast*, uniform prior $P(\text{plan}) = 1/4$ over the four plans, success probabilities $0.60, 0.70, 0.65, 0.50$ for (safe, safe), (safe, fast), (fast, safe), (fast, fast). Then $P(\text{success}) = \frac{1}{4}(0.60 + 0.70 + 0.65 + 0.50) = \frac{2.45}{4} = 0.6125$ and, by Bayes' rule, $P(\text{plan} \mid \text{success}) = \frac{1}{4} P(\text{success} \mid \text{plan})/0.6125 = (0.60, 0.70, 0.65, 0.50)/2.45 = (0.245, 0.286, 0.265, 0.204)$, so the MAP plan is (safe, fast) (`ex_planning_as_inference()`).

AI CONNECTION · Multi-Agent AI

BP needs only neighbours' messages, so if each variable belongs to a different agent it becomes a *communication protocol* that gives every agent its exact posterior without a central server. Four hypothetical fire-watch drones watch zones 1–4 on a line; $x_i \in \{0, 1\}$ (no fire / fire) and all vectors are ordered ($x_i{=}0$, $x_i{=}1$). The joint model is the pairwise tree model (7.3) with unary factors $\phi_i(x_i) = P(x_i)\, P(o_i \mid x_i)$, local prior $P(x_i{=}1) = 0.2$, and a coupling $\psi(x_i, x_{i+1}) = 3$ if the neighbouring zones agree and 1 otherwise. The readings are alarm, quiet, alarm, quiet with $P(\text{alarm} \mid \text{fire}) = 0.9$, $P(\text{alarm} \mid \text{no fire}) = 0.2$, so

$\phi_1 = \phi_3 = (0.8 \cdot 0.2,\ 0.2 \cdot 0.9) = (0.16,\ 0.18)$ and $\phi_2 = \phi_4 = (0.8 \cdot 0.8,\ 0.2 \cdot 0.1) = (0.64,\ 0.02)$. Alone, drones 1 and 3 believe $P(\text{fire}) = 0.18/0.34 = 0.529$ and drones 2 and 4 believe $0.02/0.66 = 0.030$. BP towards drone 1, then back:

$$
\begin{aligned}
m_{4\to3} &= (0.64 \cdot 3 + 0.02,\ 0.64 + 0.02 \cdot 3) = (1.94,\ 0.70),\\
\phi_3 \odot m_{4\to3} &= (0.3104,\ 0.126),\\
m_{3\to2} &= (0.3104 \cdot 3 + 0.126,\ 0.3104 + 0.126 \cdot 3) = (1.0572,\ 0.6884),\\
\phi_2 \odot m_{3\to2} &= (0.67661,\ 0.01377),\\
m_{2\to1} &= (0.67661 \cdot 3 + 0.01377,\ 0.67661 + 0.01377 \cdot 3) = (2.0436,\ 0.7179),\\
m_{1\to2} &= (0.16 \cdot 3 + 0.18,\ 0.16 + 0.18 \cdot 3) = (0.66,\ 0.70),\\
\phi_2 \odot m_{1\to2} &= (0.4224,\ 0.014),\\
m_{2\to3} &= (0.4224 \cdot 3 + 0.014,\ 0.4224 + 0.014 \cdot 3) = (1.2812,\ 0.4644),\\
\phi_3 \odot m_{2\to3} &= (0.20499,\ 0.08359),\\
m_{3\to4} &= (0.20499 \cdot 3 + 0.08359,\ 0.20499 + 0.08359 \cdot 3) = (0.6986,\ 0.4558).
\end{aligned}
$$

The beliefs $b_i = \phi_i \odot \prod_k m_{k\to i}$ are $b_1 = (0.16 \cdot 2.0436,\ 0.18 \cdot 0.7179) = (0.32697,\ 0.12922)$, $b_2 = (0.64 \cdot 0.66 \cdot 1.0572,\ 0.02 \cdot 0.70 \cdot 0.6884) = (0.44656,\ 0.00964)$, $b_3 = (0.16 \cdot 1.2812 \cdot 1.94,\ 0.18 \cdot 0.4644 \cdot 0.70) = (0.39768,\ 0.05851)$, $b_4 = (0.64 \cdot 0.6986,\ 0.02 \cdot 0.4558) = (0.44708,\ 0.00912)$, all summing to the same 0.4562. Hence $P(\text{fire}_1 \mid \text{all readings}) = 0.12922/0.4562 = 0.283$ and $P(\text{fire}_3 \mid \cdot) = 0.128$: after $2(n-1) = 6$ messages the quiet neighbours have pulled both alarms down (Fig. 7.9), to exactly the centralised posteriors.

PYTHON LAB · s07_examples.py · ex_multiagent_bp()

```
m_4->3 = [1.94 0.7], m_3->2 = [1.0572 0.6884], m_2->1 = [2.0436 0.7179],
m_1->2 = [0.66 0.7], m_2->3 = [1.2812 0.4644]
after BP P(fire) = [0.2833 0.0211 0.1283 0.02]; exact check:  [0.2833 0.0211
0.1283 0.02]
```

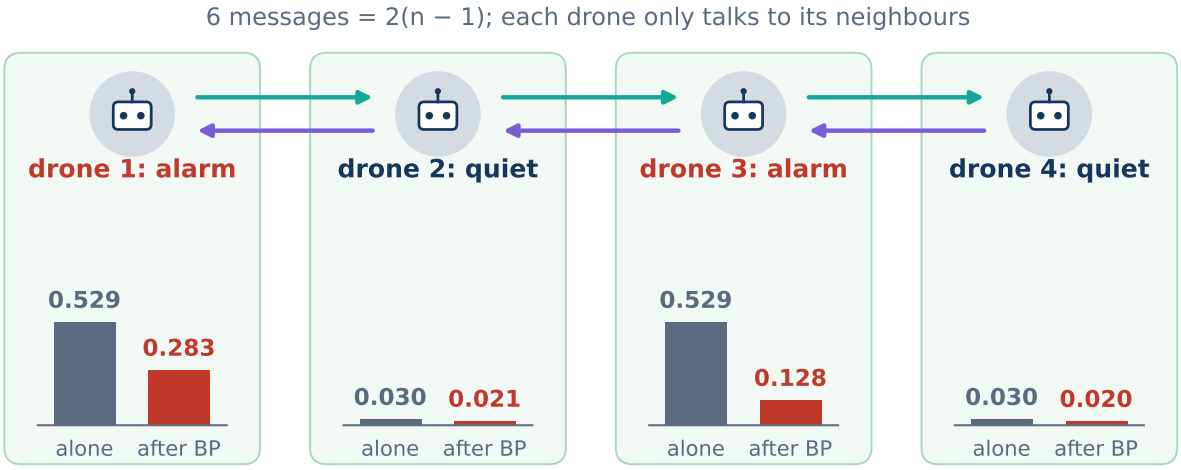


Figure 7.9. Belief propagation as a protocol between agents: each drone talks only to its neighbours, yet all obtain exact posteriors.

LINK TO COURSE SESSION 27 · Multi-Agent AI

Distributed BP underlies cooperative perception, and max-sum solves coordination graphs $\max_{\mathbf{a}} \sum_{(i,j)} Q_{ij}(a_i, a_j)$ (Session 8). On loopy communication graphs it is approximate; its double counting is a rumour echoing around a cycle of agents.

Table 7.4. This session in the Advanced AI course. Plain numbers in parentheses are course sessions; "Session k" refers to these notes.

Concept today	Where it reappears	Role there
Filtering / belief state	POMDPs (21), autonomous agents (25)	sufficient statistic for decisions
VE, evidence $P(e)$	learning PGMs (Session 10 = course session 10)	E-step and likelihood
BP, loopy BP	factor graphs (Session 8), approx. inference (Session 11), multi-agent AI (27)	message passing, coordination
Viterbi / max-product	generative models (4), planning (23–24)	decoding, most probable plan
Gaussian conditioning, Kalman	model-based RL (20), robotics	state estimation in world models
Bayes estimators	decisions (21), causal effects (Session 15)	choosing actions and estimates

PART V · AI CONNECTIONS

7.16 Summary

THINK ABOUT IT A 500-node network has treewidth 8 and a 30-node network has treewidth 25. Which is easier for exact inference, and why?

THINK ABOUT IT When would an agent prefer max-marginal decoding of a hidden sequence over the Viterbi path?

THINK ABOUT IT Loopy BP double-counts evidence. Can you describe a group of human agents who make the same mistake?

Table 7.5. Cheat sheet of key formulas.

Item	Formula
VE step and cost	$\psi = \prod_{\phi \ni Z} \phi$, $\tau = \sum_Z \psi$ (or $\max_Z \psi$); $O(n\, d^{w+1})$, $w =$ induced width
BP message	$m_{i\to j}(x_j) = \sum_{x_i} \phi_i \psi_{ij} \prod_{k \in N(i)\setminus j} m_{k\to i}$; $b_i = \phi_i \prod_k m_{k\to i}$
Forward / Viterbi	$\alpha_t(l) = B_l(x_t) \sum_k \alpha_{t-1}(k) A_{kl}$; $\delta_t(l) = B_l(x_t) \max_k \delta_{t-1}(k) A_{kl}$
Gaussian / Kalman	$\mu_a + \Sigma_{ab}\Sigma_{bb}^{-1}(x_b - \mu_b)$, $\Sigma_{aa} - \Sigma_{ab}\Sigma_{bb}^{-1}\Sigma_{ba}$; $K = \frac{P+q}{P+q+r}$
Bayes actions	squared $\to$ mean; absolute $\to$ median; 0–1 $\to$ MAP; Hamming $\to$ max-marginals

Table 7.6. Common pitfalls and how to avoid them.

Pitfall	Remedy
Summing out evidence variables	reduce every factor to $E = e$ first
"VE is exponential, so useless"	exponential in the width w, not in n; eliminate leaves first (min-fill)
Trusting loopy BP like exact BP	check convergence, damp, compare with exact results on small cases
Max-marginals as "the" explanation	use Viterbi / max-product for a coherent joint assignment

Further reading. Koller & Friedman (2009), Chs. 9–13; Bishop (2006), Ch. 8, 13; Murphy (2022); Pearl (1988); Russell & Norvig (2021), Chs. 13–14; Casella & Berger (2002), Ch. 7.

APPENDIX

7.17 Appendix: Full Inductive Proof of VE Correctness

Let $Z_1, \dots, Z_{|W|}$ be the variables of W listed in the elimination order σ, with values $z_1, \dots, z_{|W|}$. We show by induction on $k = 0, 1, \dots, |W|$ that after processing $Z_1, \dots, Z_k$, the remaining factor set Φ multiplies to $\sum_{z_1,\dots,z_k} \prod_i P(x_i \mid \mathrm{pa}(x_i))|_{E=e}$ (summed over only the first k eliminated variables, with $Z_{k+1}, \dots, Z_{|W|}$ and Y still free). **Base case** $k = 0$: Φ is exactly the CPTs restricted to evidence, whose product is $P(\mathbf{x})|_{E=e}$, matching the claim trivially. **Inductive step**: assume the claim for k; at step $k+1$ we collect every factor in Φ whose scope contains Z_{k+1} into a product ψ, and the remaining factors Φ' do not depend on Z_{k+1}. By the inductive hypothesis, the product of all factors, $\psi \cdot (\text{product of } \Phi')$, equals $\sum_{z_1,\dots,z_k} \prod_i P(x_i \mid \mathrm{pa}(x_i))|_e$. Summing both sides over z_{k+1}, the right-hand side becomes $\sum_{z_1,\dots,z_{k+1}} \prod_i P(x_i \mid \mathrm{pa}(x_i))|_e$; on the left, since the factors in Φ' do not depend on Z_{k+1} they can be pulled outside the sum by the distributive law (7.2): $\sum_{z_{k+1}} \psi \cdot \Phi' = \Phi' \cdot \sum_{z_{k+1}} \psi = \Phi' \cdot \tau$, which is exactly the product of the new factor set $\Phi' \cup \{\tau\}$ after step $k+1$. This is the claim for $k+1$. After all of W is eliminated ($k = |W|$), Φ multiplies to $\sum_w \prod_i P(x_i \mid \mathrm{pa}(x_i))|_e = P(y, e)$ by definition, completing the proof. Normalizing by $\sum_y P(y, e) = P(e)$ then yields $P(y \mid e)$.

CONNECTIONS AND TAKEAWAYS

Variable elimination gives us *exact* inference whose cost is governed entirely by the graph's treewidth — a purely structural quantity, computable before ever touching the numbers in the CPTs. This is a genuinely different kind of tool than Sessions 5–6: those sessions were about *representing* independence; this session is about *exploiting* it computationally.

But VE was presented mostly for directed Bayesian networks, using factors that happened to be conditional probability tables $P(x_i \mid \mathrm{pa}(x_i))$. The factor-graph view in the recommendation example and the pairwise potentials of the drone line hint at something more general: factors don't need a "child given parents" interpretation at all — they can just be arbitrary non-negative compatibility scores between variables that mutually constrain each other, with no natural direction of generation. **Session 8** formalizes exactly this: undirected graphical models (Markov networks) and factor graphs, needed whenever the dependencies in a system are symmetric rather than generative — and variable elimination and belief propagation, as we'll see, apply to them with almost no modification at all.